INTRODUCTION

1. INTRODUCTION

ZentroMart is a web-based electronics e-commerce platform developed using the Django framework. The system is designed to provide a convenient and secure online shopping experience for customers while enabling shop owners and delivery personnel to manage business operations efficiently. The platform offers a wide range of electronic products including smartphones, laptops, accessories, smart devices, and other digital gadgets.

The system consists of four major modules: User (Customer), Shop Owner, Delivery Boy, and Administrator. Users can create accounts, browse products, add items to their cart, place orders, make payments, track deliveries, and provide product reviews. Shop owners can manage products, categories, brands, and customer orders. Delivery personnel can view assigned deliveries and update delivery statuses. The administrator oversees the overall functioning of the platform and manages users and system data.

The project aims to automate the online shopping process, improve customer satisfaction, and provide an efficient digital marketplace for electronic products using modern web technologies.

Objective

The main objective of **ZentroMart** is to develop a web-based electronics e-commerce platform that provides customers with a convenient and secure online shopping experience. The system enables users to register, log in, manage their profiles, browse and search for electronic products, add items to their cart, place orders, track deliveries, and submit reviews and ratings. It also allows shop owners to efficiently manage products, categories, brands, inventory, and customer orders. The platform supports delivery personnel in updating and managing delivery statuses while ensuring secure storage of customer, product, and order information in a centralized database. By automating various shopping and management processes, **ZentroMart** aims to provide a responsive, user-friendly, and efficient e-commerce solution developed using the Django framework.

ABOUT THE ORGANIZATION

2. ABOUT THE ORGANIZATION

Progressive Software Solutions & Training is a fast developing, well established Information Technology enterprise with highly talented and efficient staff meeting the needs of clients throughout the world. The company is so popular that the clients feed the company with repeat orders, having faith in the capabilities. The staff includes management and engineering personnel, programmers, web developers, graphic designer, business analysts, technical writers etc. apart from the regular commercial and administrative hands. The motto of this reliable software provider is 'DO BEST; GET BEST' Progressive Software Solutions & Training committed to provide what they have promised. Progressive Software Solutions & Training strives for the satisfaction of their clients and dedicates themselves for the better services to their customers.

Progressive Software Solutions & Training are famous in providing high quality, cost cutting, requirement oriented and value added web and software solutions with remarkable cyber services. Progressive Software Solutions & Training are specialist in developing ever rewarding internet existence for ambitious companies which are in the hangover of swift moving internet facilities. Their aim is to grow with such organizations in a very fruitful manner. Their main concentration is to collect business oriented technological know-how and utilize them for the wellbeing of business concerns.

Progressive Software Solutions & Training experts in various technologies are committed to deliver effective and creative service to customers.

Services

- Software Development
- Web Development
- SEO Services (Search Engine Optimization)

SYSTEM ANALYSIS

3. SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

The existing system for purchasing electronic products mainly relies on physical stores, where customers must visit the shop to view and buy products. Product information, inventory details, and order records are often managed manually, making the process time-consuming and less efficient. Customers face difficulties in comparing products, checking availability, and tracking orders, while sellers may encounter challenges in managing inventory and customer records accurately.

Limitation: The system requires customers to be physically present at the store, reducing convenience and accessibility.

3.2 PROPOSED SYSTEM

The proposed system, ZentroMart, is a web-based electronics e-commerce platform that allows users to browse, search, and purchase products online. It provides secure user registration, product management, order processing, and delivery tracking. The system improves convenience, reduces manual work, and offers an efficient shopping experience for customers, shop owners, and delivery personnel.

Advantage: Customers can shop anytime and anywhere through the online platform.

3.3 SOFTWARE REQUIREMENTS SPECIFICATION

HARDWARE SPECIFICATION

The use of sufficient hardware is very important for the proper working of any software. Following are the hardware required by the system.

Processor	Intel Core i5
Processor Speed	1.5 GHz or higher
System Bus	32 Bit
Memory	1 GB RAM or higher
Hard Disk	40 GB or higher
Monitor	15.6” LCD Monitor
Keyboard	104 Keys

SOFTWARE SPECIFICATION

Software is a part of a computer system that consists of encoded information or computer instructions, in contrast to the physical hardware from which the system is built. Following are the software required by the system.

Operating System	Windows 11
IDE	Visual Studio code
Front End	PYTHON DJANGO, HTML 5
Back End	SQLITE3
Web Server	APACHE
Browser	Microsoft Edge, Mozilla Firefox, Chrome

3.4 FEASIBILITY STUDY

The main objective of this study is to determine whether the proposed system is feasible or mainly there are three types of feasibility study to which the proposed system is subjected to as described below:

- Economic feasibility
- Technical feasibility
- Behavioral feasibility

The proposed system must be evaluated from a technical viewpoint first, and if technically feasible then their impact on organization must be accessed. If compatible, the behavioral system can be devised. Then those must be tested for economic feasibility.

ECONOMIC FEASIBILITY

The proposed system is economically feasible as it is developed using open-source technologies such as Python, Django, and SQLite. The development cost is low, and the system reduces manual work, paperwork, and operational expenses, making it cost-effective.

TECHNICAL FEASIBILITY

The system is technically feasible because it uses modern and widely available technologies such as Django, HTML, CSS, JavaScript, and SQLite. The required hardware and software resources are easily accessible, and the system can be implemented without major technical difficulties.

BEHAVIORAL FEASIBILITY

The proposed system is user-friendly and easy to operate. Users, shop owners, delivery personnel, and administrators can easily understand and use the system with minimal training. The system improves efficiency and is expected to be readily accepted by all users.

3.5 DATA FLOW DIAGRAM

To start the system design, something analogue to the architecture blueprint as a starting point to design is required. It is a way to focus on functions rather than physical implementation. One such tool is a DFD.

Structured analysis is a set of techniques and graphical tools that help the analyst to develop a new kind of system specification that are easily understandable to the user. DFD's show the major decompositions of the system functions and their interfaces. The DFD is graphic and presents a picture of what is being specified and is conceptually easy to understand presentation of the application.

One important feature of DFD's is that it is logical rather than physical. The elements of the system do not depend on vendor or hardware. They specify in precise, concise manner the working of the system and how it hangs together.

DFD is the graphic representation of data movement process, and files used in support of an information system. There are several rules of thumb used in drawing DFDs.

- Process should be named and numbered for easy references.
- The direction of flow is from top to bottom and from left to right.
- When a process is imported in the lower level's details, they must be numbered.
- Process and data flow names have the first letter of each word must be a capital letter.

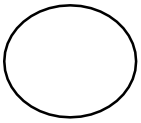
The symbol used in DFD are shown be.



Source or destination of data



Data flow

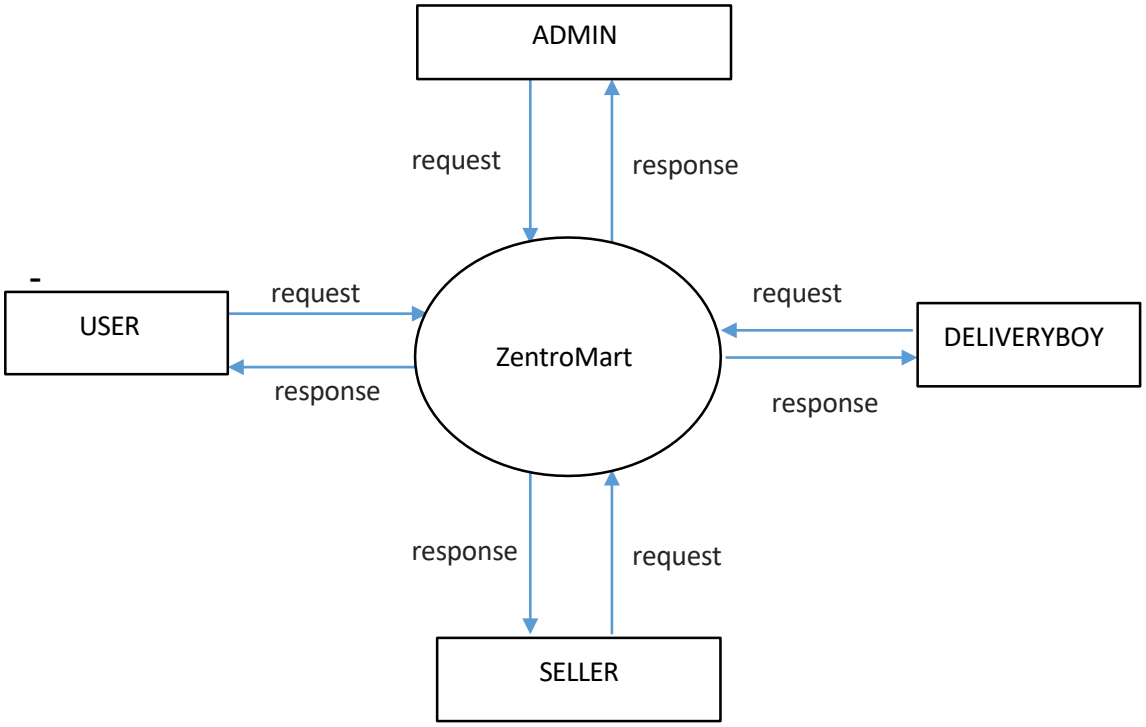


Process that transforms data flow

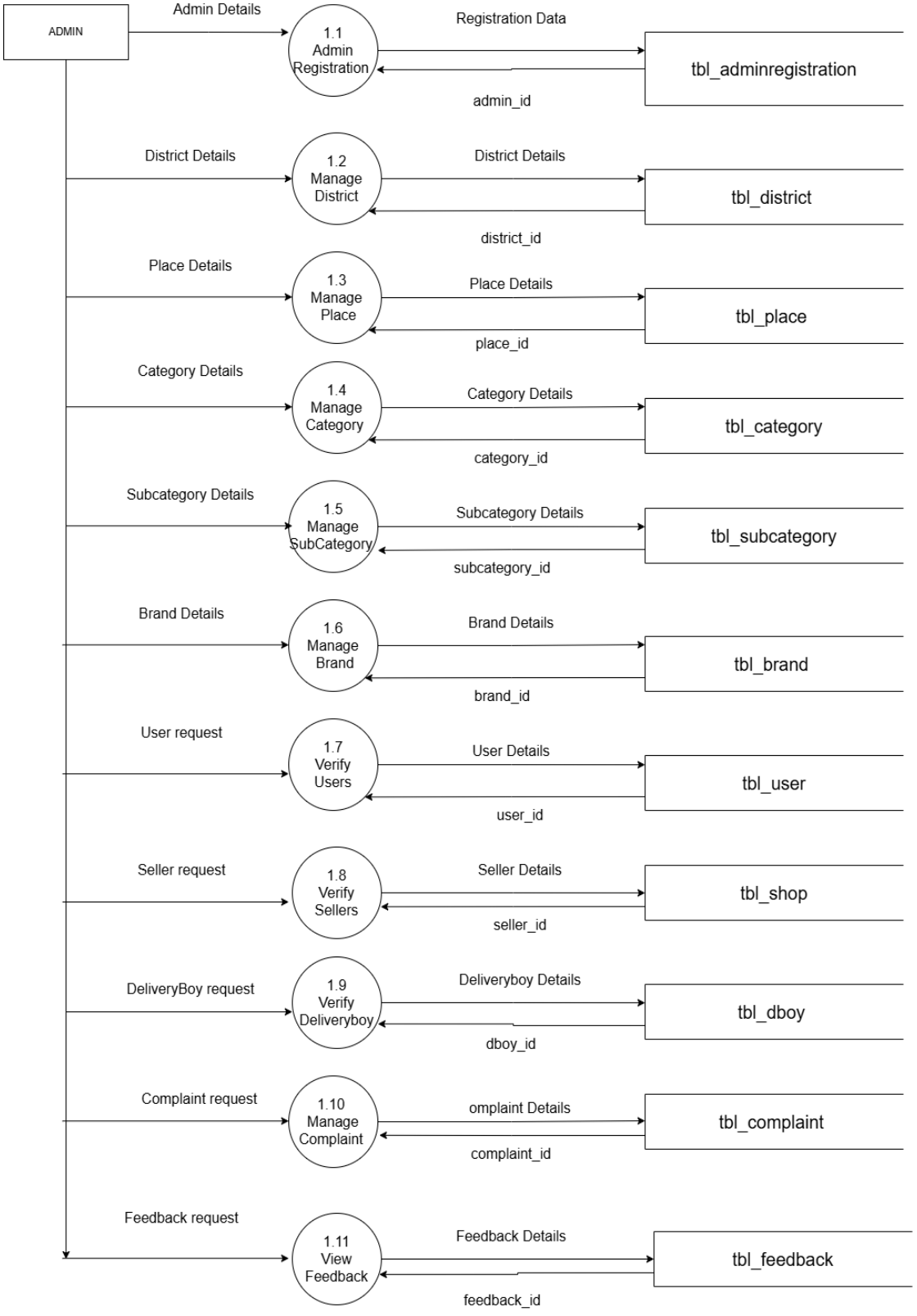


Data store

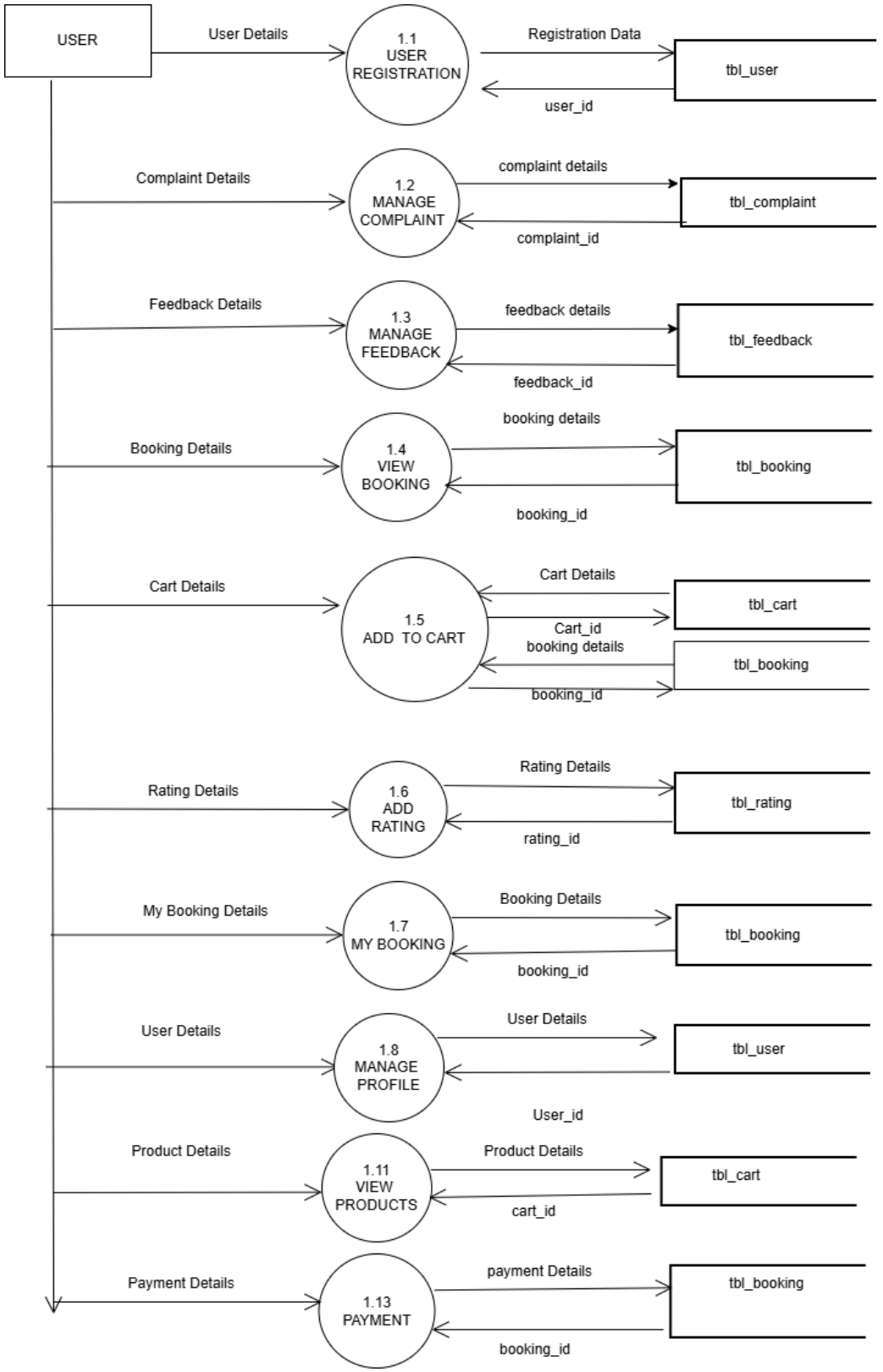
LEVEL 0


ADMIN
request
response
-
USER
request
response
ZentroMart
request
response
DELIVERYBOY
response
request
SELLER

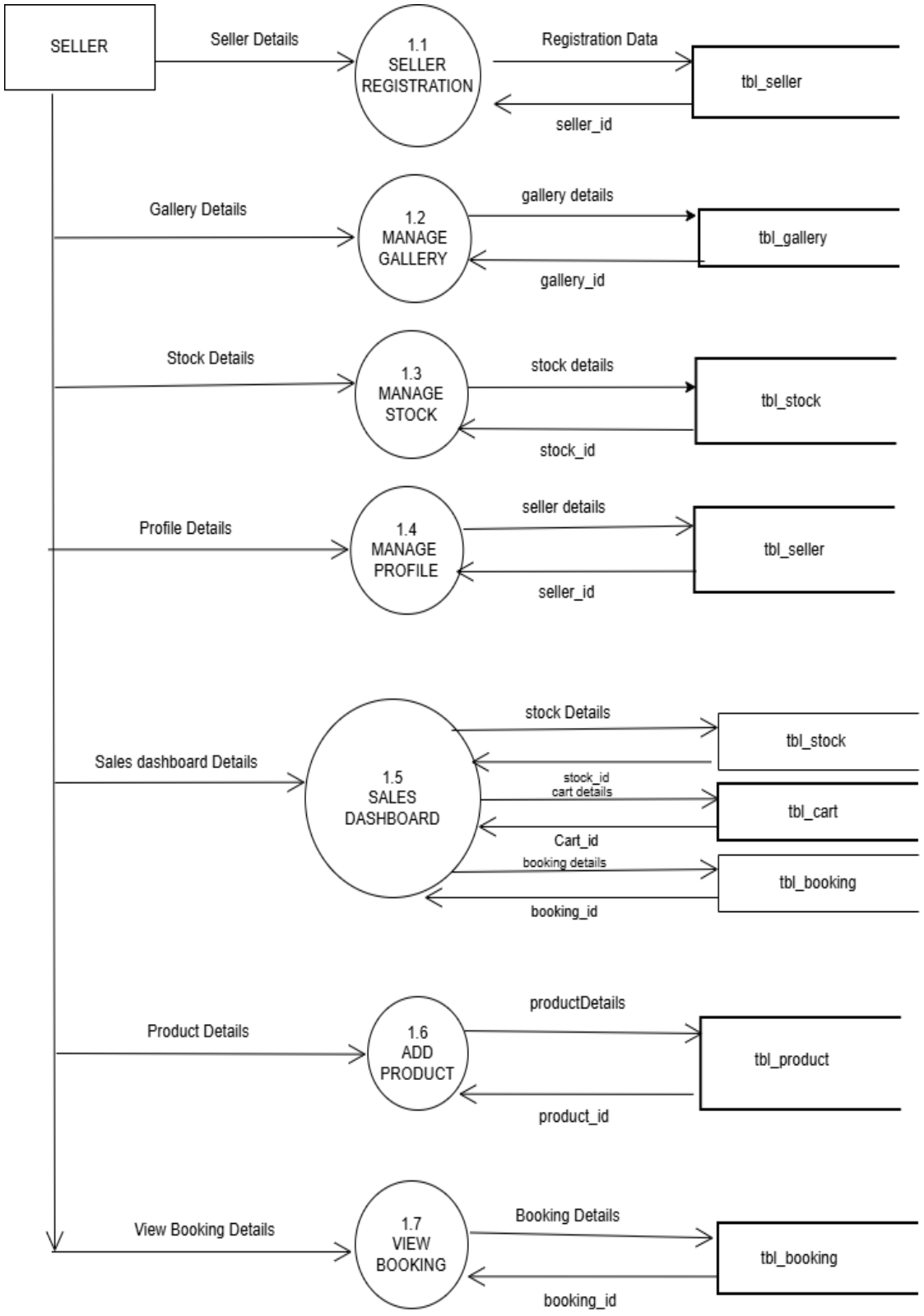
LEVEL 1 FOR ADMIN



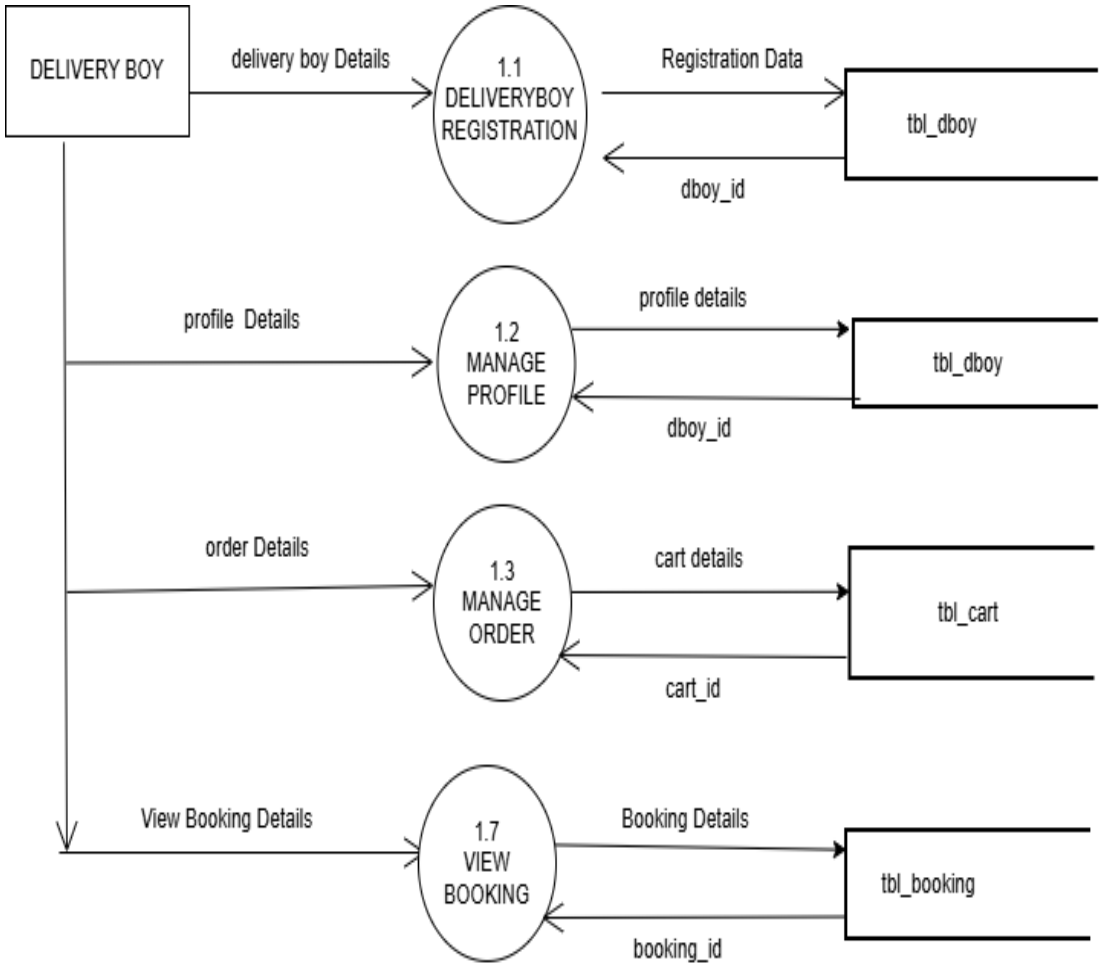
LEVEL 1 FOR USER



LEVEL 1 FOR SELLER



LEVEL 1 FOR DELIVERY BOY



SYSTEM DESIGN

4. SYSTEM DESIGN

System design is the creative act of invention developing new inputs, a database offline files, method, procedures, and output for processing business data to meet organization objectives. The design phase focuses on the detailed implementation of the system recommended in the feasibility study. The design phase is a transition from user-oriented document to a document oriented to the programmers or database personnel. Characteristics of a well-designed system are:

- Accessibility
- Decision making ability.
- Economy
- Flexibility
- Reliability
- Simplicity

The design will determine the success of the system. System design is based on the information gathered during system analysis. System design goes through two phases of development.

- Logical design- DFD shows the logical flow of a system and defines the boundaries of the system. For the candidate the system it describes the input, outputs, databases, and procedures-all in a format that meets the user's requirements.
- Physical design - This produces the working system by defining the design specification that tells programmers exactly what the candidate system must do.

4.1 INPUT DESIGN

Input design is the link that ties the information system into the world of its users. The input design involves determining what the inputs are, how the data should be performed, how to validate data, how minimize data entry and how to provide a multi-user facility inaccurate.

Input data are the most common cause of error in data processing. Errors entered by data entry operators can be controlled by input design. Input design is the process of converting user.

Originated inputs to a computer-based format. Input data are collected and organized into groups of similar data. Once identified, appropriate input media are selected for processing. All the input data validated in the order and if any data violates any condition, the user is warned by a message. If the data satisfies all the conditions, then it is transferred to the appropriate tables in the database.

4.2 OUTPUT DESIGN

Output design involves specifying how production of on-screen reports and page-based reports will occur. Output may occur to database or file for storing information entered or also for use by other systems. Computer output is the most important and direct source of information to the user. Output design is a very important phase because the output needs to be in an attractive manner. Efficient, intelligible output design improves the system relationship with the user and help in decision making. A major form of the output is the hard copy from the printer and screen reports. Printouts are designed around the output requirements of the user. Allowing the user to view to the sample screen is important because the user is ultimate judge of the quality of output. The output model of this system is the user-friendly window. These user-friendly windows are meant for the purpose of easy view of the stored information.

4.3 TABLE DESIGN

A database is a collection of data. Database design refers to the design of the tables used to store data. The database involves name of records, data item with its name, type and size. In the design of the database program first we must thoroughly look into the requirements of the program for the design of database. Then we must design how much tables are required in the database. Thereafter as per the requirement of the end users we can decide which fields that must be in this table. As per a general rule a provision must be taken in the design for the future enhancement of the program. Some of important tables are:

TABLE STRUCTURE

1. **Table name: tbl_adminregistration**
   Table description: To store the details of admin.

Field Name	**Data Type**	**Constraints (Primary key, foreign key, not null, null)**	**Field Descrition**
adminregistration_id	int (10)	Primary key	Admin id
adminregistration _name	varchar (30)	Not null	Name of the admin
adminregistration_password	varchar (30)	Not null	Password of the admin
adminregistration _email	varchar (50)	Not null	Email of the admin

2. **Table name: tbl_district**
   Table description: To store the details of district

Field Name	**Data Type**	**Constraints (Primary key, foreign key, not null, null)**	**Field Description**
district_id	int(10)	Primary key	District id
district_name	varchar(30)	Not null	Name of the district

3. **Table name: tbl_place**

   Table description: To store the details of place

Field Name	**Data Type**	**Constraints (Primary key, foreign key, not null, null)**	**Field Description**
place_id	int(10)	Primary key	Place id
place_name	varchar(30)	Not null	Name of the place
district_id	int(10)	Foreign key (tbl_district)	District id
Place_pin	Varchar(30)	Not null	Pincode of the place

4. **Table name: tbl_category**

   Table description: To store the details of category of the product

Field Name	**Data Type**	**Constraints (Primary key, foreign key, not null, null)**	**Field Description**
category_id	int(10)	Primary key	Category id
Category_name	varchar(30)	Not null	Name of Category

5. **Table name: tbl_subcategory**

Table description: To store the details of subcategory of the product.

Field Name	**Data Type**	**Constraints (Primary key, foreign key, not null, null)**	**Field Description**
subcategory_id	int(10)	Primary key	subcategory id
subcategory_name	varchar(30)	Not null	Name of subcategory
Category_name_id	varchar(30)	Foreign key	Category_id

6. **Table name: tbl_brand**

Table description: To store the details of category of the product

Field Name	**Data Type**	**Constraints (Primary key, foreign key, not null, null)**	**Field Description**
brand_id	int(10)	Primary key	Brand id
brand_name	varchar(30)	Not null	Name of brand

7. Table name: tbl_user

Table description: To manage user details.

Field Name	Data Type	Constraints (Primary key, foreign key, not null, null)	Field Description
user_id	int(11)	Primary key	user Id
user _name	varchar(25)	Not null	Name of user
user _email	varchar(25)	Not null	Email of user
user _password	varchar(25)	Not null	Password of user
user _gender	varchar(20)	Not null	Gender of user
user _contact	varchar(300)	Not null	Contact of user
user _address	varchar(300)	Not null	Address of the user
user _photo	varchar(300)	Not null	Photo of user
user _proof	varchar(300)	Not null	Proof of user
user _status	varchar(30)	Not null	Status of user
place_id	int(10)	Foreign key (tbl_place)	Place id

8. Table name: tbl_feedback

Table description: To manage user feedback details.

Field Name	Data Type	Constraints (Primary key, foreign key, not null, null)	Field Description
feedback_id	int(10)	Primary key	Feedback id
feedback_content	varchar(300)	Not null	Content of feedback
feedback_date	varchar(50)	Not null	Date of feedback
user_id	int(10)	Foreign key (tbl_user)	User Id

9. Table name: tbl_complaint

Table description: To manage user complaint details.

Field Name	Data Type	Constraints (Primary key, foreign key, not null, null)	Field Description
complaint_id	int(10)	Primary key	Complaint Id
complaint_title	varchar(250)	Not null	Title of the complaint
complaint_content	varchar(250)	Not null	Content of the complaint
complaint_date	date	Not null	Date of the complaint
user_id	int(10)	Foreign key (tbl_user)	User Id
complaint_reply	varchar(25)	Not null	Reply of the complaint
complaint_status	varchar(25)	Not null	Status of the complaint

10. Table name: tbl_booking

Table description: To manage user booking details.

Field Name	Data Type	Constraints (Primary key, foreign key, not null, null)	Field Description
booking_id	int(10)	Primary key	Booking id
booking_date	varchar(300)	Not null	Date of booking
booking_amount	Float(50)	Not null	Amount of booking
booking_status	Int(10)	Not null	Status of booking
booking_deliveraddress	Varchar(20)	Not null	Deliveraddress of booking
user_id	int(10)	Foreign key (tbl_user)	User Id

11. Table name: tbl_cart

Table description: To store the details of the cart of a user.

Field Name	Data Type	Constraints (Primary key, foreign key, not null, null)	Field Description
cart_id	int(11)	Primary key	Cart Id
cart_quantity	int(25)	Not null	Quantity of cart
cart_status	int(25)	Not null	Status of cart
productid	int(25)	Foreign key (tbl_product)	Product id
bookingid	int(20)	Foreign key (tbl_booking)	Booking id
deliveryboy	int(300)	Foreign key (tbl_dboy)	Deliveryboy id

12. Table name: tbl_rating

Table description: To store the details of rating a product by user.

Field Name	Data Type	Constraints (Primary key, foreign key, not null, null)	Field Description
rating_id	int(10)	Primary key	Rating Id
rating_data	Int(10)	Not null	Data of rating
User_id	Int(10)	Foreign key (tbl_user)	User Id
user_review	time	Not null	Review of user
datetime	datetime	Not null	Datetime of rating
product	int(10)	Foreign key (tbl_product)	Product id

13. Table name: tbl_shop

Table description: To store shop details

Field Name	Data Type	Constraints (Primary key, foreign key, not null, null)	Field Description
shop_id	int(10)	Primary key	Shop Id
shop _name	varchar(25)	Not null	shop name
shop _contact	varchar(25)	Not null	Shop email
shop_email	varchar(25)	Not null	Password Of shop
shop _password	varchar(100)	Not null	Address of shop
shop_address	int(25)	Not null	Contact of shop
shop_photo	varchar(25)	Not null	Photo of shop
shop _proof	varchar(25)	Not null	Id proof of shop
shop _status	int(10)	Not null	Status of shop
place_id	varchar(30)	Foreign key (tbl_place)	Place_id

14. Table name: tbl_gallery

Table description: To store photo of the product.

Field Name	Data Type	Constraints (Primary key, foreign key, not null, null)	Field Description
gallery_id	int(10)	Primary key	Gallery_Id
gallery_file	file	Not null	File of gallery
Product_id	date	Foreign key (tbl_product)	Product_id

15. Table name: tbl_product

Table description: To store product details.

Field Name	Data Type	Constraints (Primary key, foreign key, not null, null)	Field Description
traffic_id	int(10)	Primary key	Participant Id
traffic _photo	varchar(300)	Not null	Traffic photo
traffic _content	varchar(300)	Not null	Details of traffic
traffic _datetime	datetime	Not null	Time of safetybutton
user_id	int(10)	Foreign key (tbl_user)	User Id
place_id	int(10)	Foreign key (tbl_place)	Place id

16. Table name: tbl_stock

Table description: store the stock details.

Field Name	Data Type	Constraints (Primary key, foreign key, not null, null)	Field Description
stock_id	int(10)	Primary key	Stock Id
Stock_quantity	varchar(300)	Not null	Quantity of stock
Stock_date	date	Not null	Date of the stock
product_id	int(10)	Foreign key (tbl_product)	Product Id

17. Table name: tbl_dboy

Table description: To store delivery boy details

Field Name	Data Type	Constraints (Primary key, foreign key, not null, null)	Field Description
dboy_id	int(10)	Primary key	dboy Id
dboy _name	varchar(25)	Not null	dboy name
dboy _contact	varchar(25)	Not null	Dboy email
dboy_email	varchar(25)	Not null	Password Of dboy
dboy _password	varchar(100)	Not null	Address of dboy
dboy_address	int(25)	Not null	Contact of dboy
dboy_photo	varchar(25)	Not null	Photo of dboy
dboy _proof	varchar(25)	Not null	Id proof of dboy
dboy _status	int(10)	Not null	Status of dboy
place_id	varchar(30)	Foreign key (tbl_place)	Place_id

4.4 PROCESS DEIGN

There are 4 modules in the system.

1. Admin
2. Seller
3. User
4. Delivery boy

1. ADMIN

- Add district

  Administrator has the duty to add district.

- Add place

  Administrator has the duty to add place.

- Add category,subcategory

  Administrator has the duty to add category,subcategory

- Verify user and deliveryboy,seller

  Administrator has the duty to accept and reject.

- Manage complaint,feedback

  Administrator has the duty to manage the complaint and feedback.

2. Seller

- Add product

  Seller has the duty to add products for shops.

- Manage booking

  Seller has the duty to manage booking .

- Add gallery

  Seller has the duty to add gallery for shops.

- Add stock

  Seller has the duty to add stock for shops.

3. USER

- User can login

  All user can login.
- Viewproduct list

  User can view the product list.
- Mybooking

  User can manage mybooking.
- My cart

  User can add products to mycart.
- Add feedback

  User can add feedback.
- Add complaint

  User can add product complaints.
- Add rating

  User can add product rating.

4. Deliveryboy

- Deliveryboy can login

  All delivery boy can login.
- My order

  Deliveryboy can manage the order.
- View booking

  Deliveryboy can view the booking by the user

SYSTEM TESTING AND IMPLEMENTATION

5. SYSTEM TESTING & IMPLEMENTATION

Testing is the process of examining the software to compare the actual behaviour with that of the excepted behaviour. The major goal of software testing is to demonstrate that faults are not present. In order to achieve this goal the tester executes the program with the intent of finding errors. Though testing cannot show absence of errors but by not showing their presence it is considered that these are not present.

System testing is defined as the process by which one detects the defects in the software. Any software development organization or team has to perform several processes. Software testing is one among them. It is the final opportunity of any programmer to detect and rectify any defects that may have appeared during the software development stage. Testing is a process of testing a program with the explicit intention of finding errors that makes the program fail. In short system testing and quality assurance is a review in software products and related documentation for completion, correctness, reliability, and maintainability.

System testing is the first stage of implementation, which is aimed at ensuring that the system works accurately and efficiently before live operation commences. Testing is vital to the success of the system. System testing makes a logical assumption that if all the parts of the system are correct and the goal will be successfully achieved. A series of testing are performed for the proposed system before the proposed system is ready for user acceptance testing.

The testing steps are:

- Unit Testing
- Integration Testing
- Validation Testing
- Output Testing
- Acceptance Testing

System Testing provides the file assurance that software once validated must be combined with all other system elements. System testing verifies whether all elements have been combined properly and that overall system function and performance is achieved. After the integration of modules, the validation test was carried out over the system. It was found that all the modules work well together and meet the overall system function and performance.

- ## UNIT TESTING

Unit testing is carried out screen-wise, each screen being identified as an object. Attention is diverted to individual modules, independently to one another to locate errors. This has enabled the detection of errors in coding and logic. Various test cases are prepared. For each module these test cases are implemented and it is checked whether the module is executed as per the requirements and outputs the desired result. In this test each service input and output parameters are checked. In unit testing,

- Module interface was tested to ensure that information properly flows into and out of the program under test.
- Boundary condition was tested to ensure that module operates properly at boundaries established to limit or restrict processing.
- All independent paths through the control structures were executed to ensure that all statements in the modules have been executed at least once.
- Error handling paths were also tested.

- ## INTEGRATION TESTING

Integration testing is a systematic technique for constructing the program structure at the same time conducting tests to uncover errors associated with interfacing. Unit tested module were taken and a single program structure was built that has been dictated by the design. Incremental integration has been adopted here.

Integration testing is a systematic technique for constructing the program structure at the same time conducting tests to uncover errors associated with interfacing. Unit tested module were taken and a single program structure was built that has been dictated by the design. Incremental integration has been adopted here.

Integration testing is a systematic technique for constructing the program structure at the same time conducting tests to uncover errors associated with interfacing. Unit tested module were taken and a single program structure was built that has been dictated by the design. Incremental integration has been adopted here. The modules are tested separately for accuracy and modules are integrated together using bottom-up integration i.e., by integrating from moving from bottom to the top. The system is checked and errors found during integration are rectified. In this testing, all the individual modules were combined, and the module wise shifting was verified to be alright. The entire software was developed and tested in small segments, where errors were easy to locate and rectify. Program builds (group of modules) were constructed corresponding to the successful testing of user interaction, data manipulation analysis, and display processing and database management.

- ## VALIDATION TESTING

Validation testing is done to ensure complete assembly of the error free software. Validation can be termed successful only if it functions in manner. Reasonably expected by the customer under validation is alpha and beta testing. Alpha testing is where the end user tests the system rather than the developer, but in a controlled environment. The software is used on a natural setting with the developer monitoring the user using the system. The developer records the errors and usage problems encountered by the user. Alpha testing is simulated or actual operational testing by potential users/customers or an independent test team at the developers' site. Alpha testing is often employed for off-the- shelf software as a form of internal acceptance testing, before the software goes to beta testing. Beta testing comes after alpha testing. Versions of the software, known as beta versions, are released to a limited audience outside of the programming team. The software is released to groups of people so that further testing can ensure the product has few faults or bugs.

Sometimes, beta versions are made available to the open public to increase the feedback field to a maximal number of future users. The user person conducts beta testing at one more site. The developer is not present during these tests. Hence, beta test can be said as the live application of software on an environment that cannot be controlled by the developer. The salesperson takes down the problems encountered during beta testing and reports to the developer at regular intervals. The developer makes suitable modifications to the software henceforth. The client-side validation is done in this testing phase. It is checked whether the data passed to each service is valid or not. Entering incorrect values in the field of email id, passwords, mobile number and name are checked in this project. If any, those values are discarded, and a message will be appeared on the screen. Like this the errors are rectified.

- **OUTPUT TESTING**

After performing the validation testing the next step is output testing of the proposed system, since no system could be useful if it does not produce the required output in a specific format. Asking the users about the format required by them, testing the output generated are considered into two ways. One is onscreen and another is printed format. The output format on the screen was found to be correct as the format was designed in the system design phase according to the user needs. For the hard copy also, the output comes out as specified requirement by the user. Hence output testing does.not result in any correction in the system.

"ZentroMart" is developed based on the user choice. It is user friendly. The output format is very clear to user.

- ## ACCEPTANCE TESTING

Acceptance testing generally involves running a suite of tests on the completed system. Each individual test, known as a case, exercises a particular operating condition of the user's environment or feature of the system, and will result in a pass or fail, or Boolean, outcome.

Acceptance testing is the user testing. I.e., whether the user is accepted or rejected the system. “ZENTROMART” is developed for peoples. Its screens are created as the base of user's choice. ZENTROMART, it helps to restore and preserve rivers through community-led cleanups. The software passed the acceptance test correctly. Screens are created as the base of user's choice.

- ## DEBUGGING

Debugging is the process of detecting and removing of existing and potential errors (also called as ‘bugs’) in a software code that can cause it to behave unexpectedly or crash. To prevent incorrect operation of a software or system, debugging is used to find and resolve bugs or defects. When various subsystems or modules are tightly coupled, debugging becomes harder as any change in one module may cause more bugs to appear in another. Sometimes it takes more time to debug a program than to code it.

To debug a program, user has to start with a problem, isolate the source code of the problem, and then fix it. A user of a program must know how to fix the problem as knowledge about problem analysis is expected. When the bug is fixed, then the software is ready to use. Debugging tools (called debuggers) are used to identify coding errors at various development stages. They are used to reproduce the conditions in which error has occurred, then examine the program state at that time and locate the cause. Programmers can trace the program execution step-by-step by evaluating the value of variables and stop the execution wherever required to get the value of variables or reset the program variables. Some programming language packages provide a debugger for checking the code for errors while it is being written at run time.

Here is the debugging process:

1. Reproduce the problem.
2. Describe the bug. Try to get as much input from the user to get the exact reason.
3. Capture the program snapshot when the bug appears. Try to get all the variable values and states of the program at that time.
4. Analyse the snapshot based on the state and action. Based on that try to find the cause of the bug.
5. Fix the existing bug, but also check that any new bug does not occur.

IMPLEMENTATION

The implementation is the final state, and it is an important phase. It involves the individual programming; system testing, user training and the operational running of developed proposed system that constitutes the application subsystems. A major task of preparing for implementation is education of users, which should really have been taken place much earlier in the project when they were being involved in the investigation and design work. During the implementation phase system actually takes physical shape. To develop a system implemented planning is very essential.

The implementation phase of the software development is concerned with translating design specification into source code. The user tests the developed system and changes are made according to their needs. Our system has been successfully implemented. Before implementation several tests have been conducted to ensure that no errors are encountered during the operation. The implementation phase ends with an evaluation of the system after placing into the operation for a period.

The process of putting the developed system in actual use is called system implementation. This includes all those activities that take place to convert from old system to new system. The system can be implemented only after testing is done and is found to be working to specifications. The implementation stage is a systems project. The implementation stage involves following tasks:

IMPLIMENTATION PLANS:

- Test system with sample data
- Detection and correction of errors
- Make necessary changes in the system.
- Check with the existing system.
- Installation of hardware and software utilities

  Training and involvement of user personnel

SECURITY TECHNOLOGIES AND POLICIES

.

6. SECURITY TECHNOLOGIES AND POLICIES

SYSTEM SECURITY

Threats to System Security

Any act or object that poses a danger to computer assets is known as threats. Management must be aware of the various kinds of threats facing the organization. By examining each threat category, the management can effectively protect the assets through policy, training and technology.

A threat can be either "intentional" (i.e., hacking: an individual cracker or a criminal organization) or "accidental" (e.g., the possibility of a computer malfunctioning, or the possibility of a natural disaster such as an earthquake, a fire, or a tornado) or otherwise a circumstance, capability, action, or event. There are several types of threats.

- Attempt to access websites and modify or destroy its contents.
- Send malicious programs such as viruses, worms and Trojans to a web server by a browser.
- Earthquake
- Excess electric supply
- Natural disasters.

SECURITY TECHNOLOGIES AND POLICIES

Maintenance involves the software industry captive, typing up system resources, it means restoring something to its original condition. Maintenance follows conversion to the extend. That change is necessary to maintain satisfactory operations relative to changes in the user's environment. Maintenance often includes minor enhancements or corrections to problems that surface in the system's operation. Maintenance is also done based on fixing the problems reported, changing the interface with other software or hardware enhancing the so its software. Any system developed should be secured and protected against possible hazards. Security measures are provided to prevent.

unauthorized access of the database at various levels. An uninterrupted power supply should be so that the power failure or voltage fluctuations will not erase the data in the files. Password protection and simple procedure to prevent the unauthorized access are provided to the users. The system allows the user to enter the system only through proper username and password.

SYSTEM SECURITY MEASURES

Security for e-commerce means complying with several key requirements, while making sure that online security measures are adopted. Having the right security measures in place is essential to maintain customer trust and confidence, and to ensure customer retention and loyalty.

To protect the confidential data from intruders, there are several security measures available today.

1. Passwords

The most inexpensive and perhaps the easiest method of security is achieved through password protection. A password is a secret word or string of characters that is used for user authentication to prove identity, or for access approval to gain access to a resource. Now passwords are commonly used by people to login to a computer system, access ATMs, etc. A password is also known as a passkey. It serves as a security measure against unauthorized access to data. Password security adds no cost to the network and is potentially a useful security measure.

MAINTENANCE

7. MAINTENANCE

Maintenance is the process of modifying a software system or component a after its delivery in order to correct faults, improve the performance and other attributes or to adapt to the changed environment. In **"ZENTROMART"** maintenance covers a wide range of activities including correcting the coding and design errors, updating the documentation and test data and upgrading the user support. Maintenance involves the software industry captive, typing up system resources. It means restoring something to its original condition. Maintenance follows conversion to the extend. The change is necessary to maintain satisfactory operations relative to changes in the user's environment. Maintenance often includes minor enhancements or corrections to problems that surface in the system's operations. Maintenance is also done based on fixing the problems reported, changing the interface with other software or hardware enhancing the software. Hardware also requires periodic maintenance to keep the system in to its standards.

The future updates and maintenance can be done in this system even if it is delivered to the user. By modifying system after it has been put into use. Modifies the existing components and adds new components to the system. Maintenance is not only bug fixing but also adapting the software to changing requirements, changing environment etc.

Software maintenance is a very broad activity that includes error correction, enhancements of capabilities, deletion of obsolete capabilities, and optimization. Because change is inevitable, mechanisms must be developed for evaluating, controlling, and making modifications. So, any work done to change the software after it is in operation is maintenance work. The purpose is to preserve the value of software over time. The value can be enhanced by expanding the customer base, meeting additional requirements, becoming easier to use, more efficient and employing newer technology.

The categories of maintenance are:

- Corrective Maintenance
- Adaptive Maintenance
- Perfective Maintenance

CORRECTIVE MAINTENANCE

This refers to modifications initiated by defects in the software. A defect can result from design errors, logic errors and coding errors. Design errors occur when changes made to software are incorrect, incomplete, wrongly communicated or the change request is misunderstood. Logic errors result from invalid tests and conclusions, incorrect implementation of design specifications, faulty logic flow or incomplete test data. Coding errors are caused by implementation of detailed logic design and incorrect use of the source code logic. Defects are also caused by data processing errors and system performance errors.

In the event of system failure due to an error, actions are taken to restore the operation of the software system. Due to pressure from management, maintenance personnel sometimes resort to emergency fixes known as "patching". The adhoc nature of this approach often gives rise to a range of problems that include increased program complexity and unforeseen ripple effects. Unforeseen ripple effects imply that a change to one part of a program may affect other sections in an unpredictable manner, thereby leading to distortion in the logic of the system. This is often due to lack of time to carry out through "impact analysis" before effecting the change.

ADAPTIVE MAINTENANCE

It includes modifying the software to match changes in the ever-changing environment. The term environment in this context refers to the totality of all conditions and influences which act from outside upon the software. A change to the whole or part of this environment will require a corresponding modification of the software.

Thus, this type of maintenance includes any work initiated as consequences of moving the software to a different hardware or software platform compiler, operating system, or new processor. Any change in the government policy can have far-reaching ramifications on the software.

PERFECTIVE MAINTENANCE

It means improving processing efficiency or performance or restricting the software to improve changeability. When the software becomes useful, the user tends to experiment with new cases beyond the scope for which it was initially developed. Expansion in requirements can take the form of enhancement of existing system functionality or improvement in computational efficiency.

Hence, perfective maintenance refers to enhancements: making the process better, faster, smaller, better documented, cleaner structured, with more functions or reports.

CONCLUSION

8. CONCLUSION

8.1 Scope for Future Enhancement

The **ZentroMart** project successfully developed an intelligent e-commerce platform that provides a complete online shopping environment for customers, sellers, delivery personnel, and administrators. The system simplifies various activities such as product management, order processing, inventory control, and delivery management through an easy-to-use web interface. A key feature of the system is the integration of **Machine Learning-based Demand Prediction**, which helps forecast future product demand using historical sales data. This enables better inventory planning, reduces stock shortages, and improves business efficiency. Developed using **Python, Django, HTML, CSS, JavaScript, and SQLite**, the system offers a secure, reliable, and scalable solution for modern online shopping requirements.

Although the current system satisfies the major requirements of an e-commerce platform, there is significant scope for future enhancement. Online payment gateway integration can be added to support digital transactions through UPI, credit/debit cards, and digital wallets. Mobile applications for Android and iOS can improve accessibility and user convenience. Advanced AI-based recommendation systems can be implemented to provide personalized product suggestions. Real-time GPS-based order tracking and AI-powered chatbot support can further enhance customer satisfaction. The demand prediction module can be improved using advanced machine learning and deep learning techniques for more accurate forecasting. Additional features such as multi-vendor marketplace support, inventory automation, multi-language support, and cloud deployment can increase the scalability and functionality of the system.

In conclusion, ZentroMart demonstrates how web technologies and machine learning can be effectively integrated to create a smart and efficient e-commerce platform. With future enhancements, the system can evolve into a more advanced, intelligent, and scalable solution capable of meeting the growing demands of the e-commerce industry.

BIBLIOGRAPHY

9.BIBLIOGRAPHY

[1] Award, Elias M. ”System Analysis and Design”, Galgotia Publications Pvt Ltd,Year:2002,Edition 2.

[2] Silberschatz, Abraham, F, Henry and “Database System Concepts”, McGraw-Hill Education,Year:1986,Edition 6

[3] Aggarwal, K K, and Singh,Yogesh “Software Engineering” ,New Age International,Year:2005,Edition

REFERENCES

1. www.google.com
2. www.wikipedia.com
3. www.python.org
4. www.freecodecamp.org
5. www.w3schools.com
6. https://stacksoverflow.com

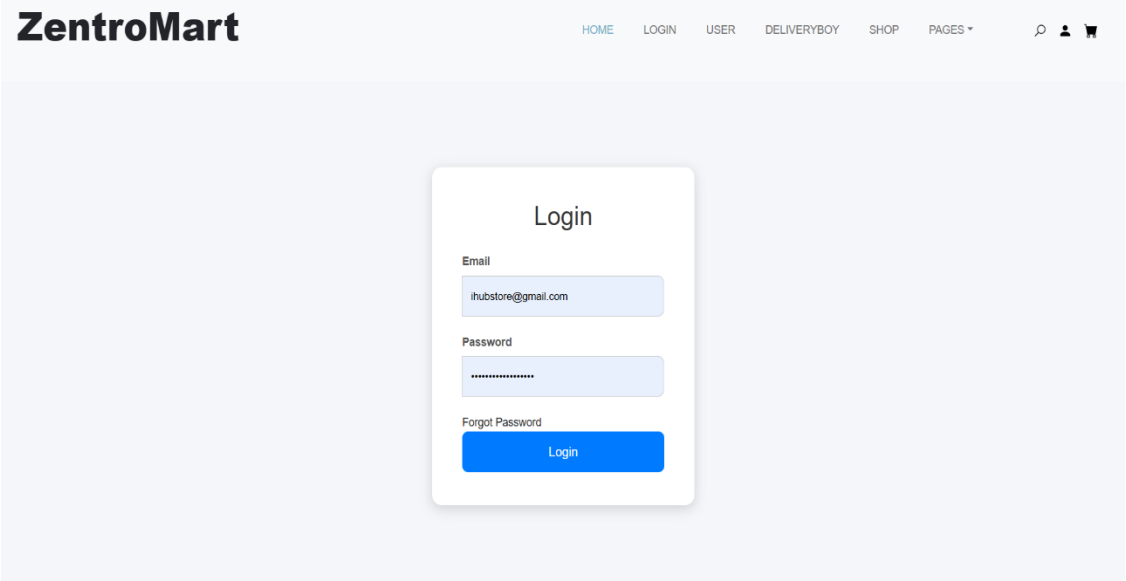
APPENDIX

10. APPENDIX

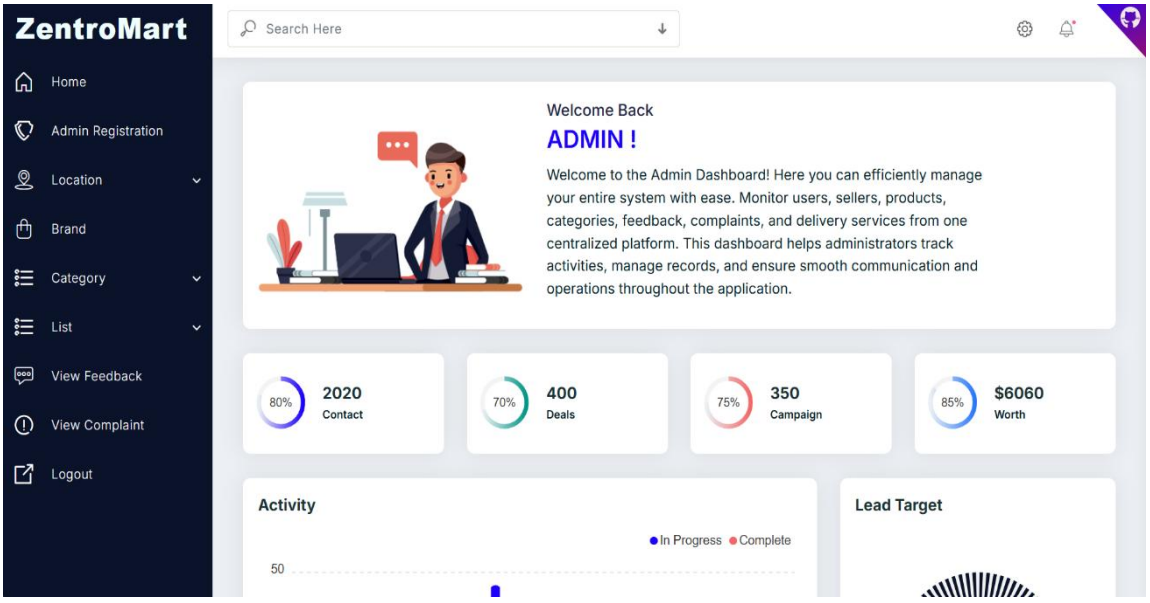
10.1 Screenshort

ADMIN

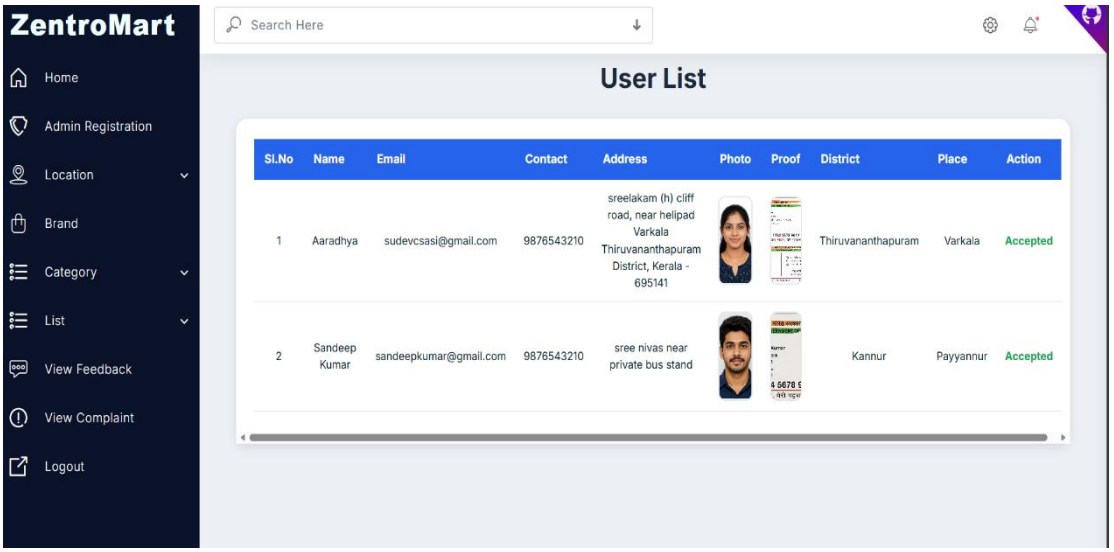
Login Page:



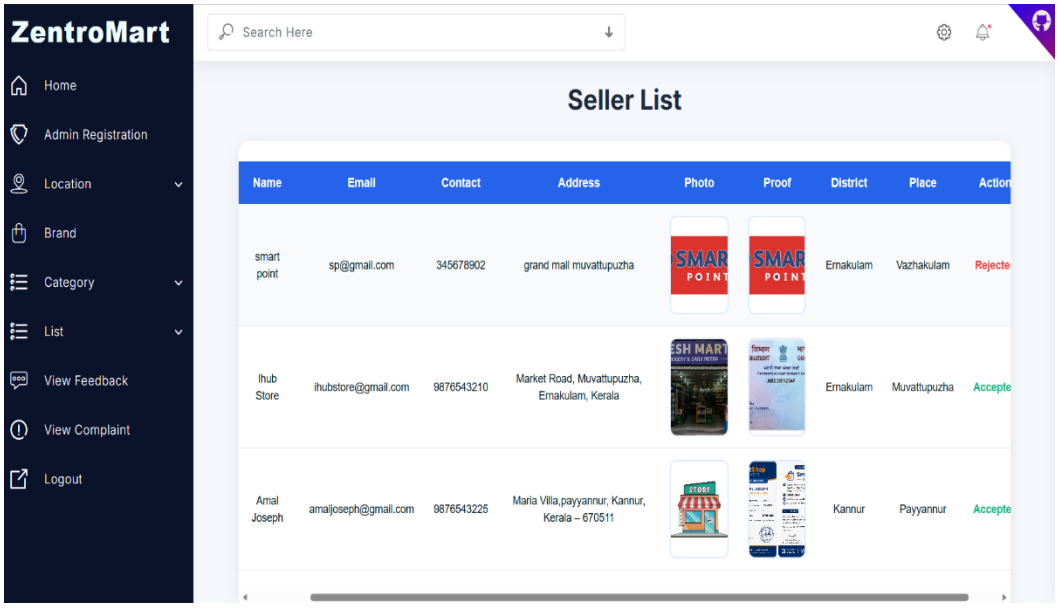
Home Page:



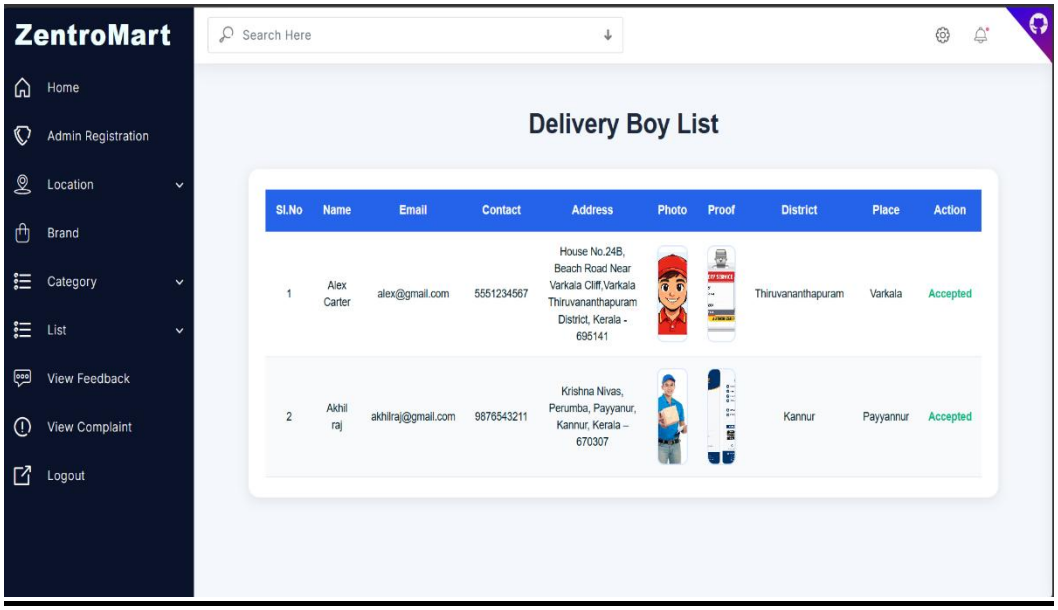
User List:



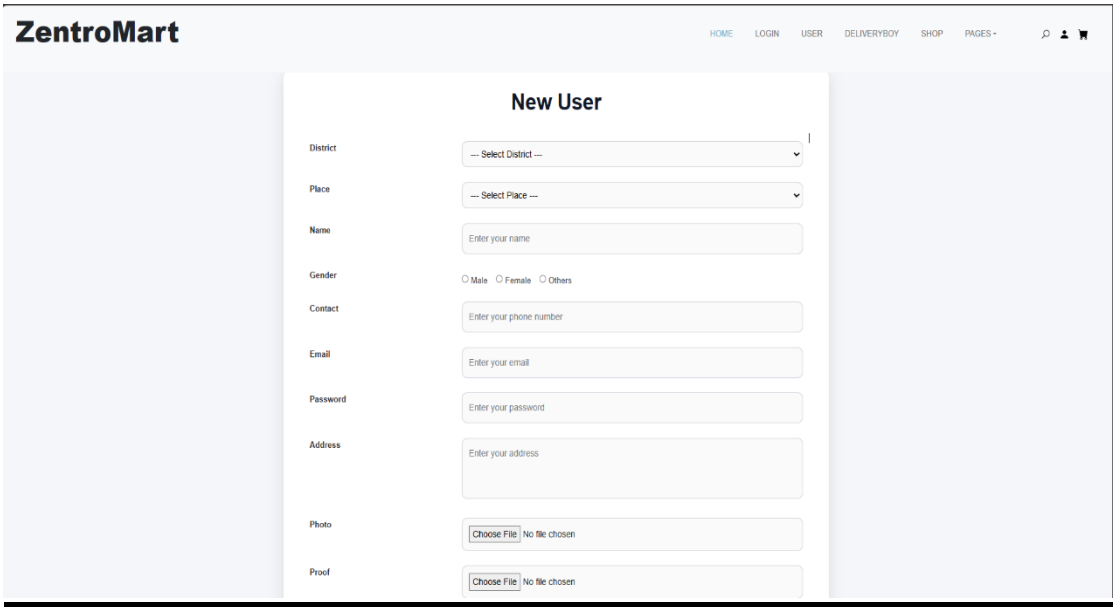
Seller List:



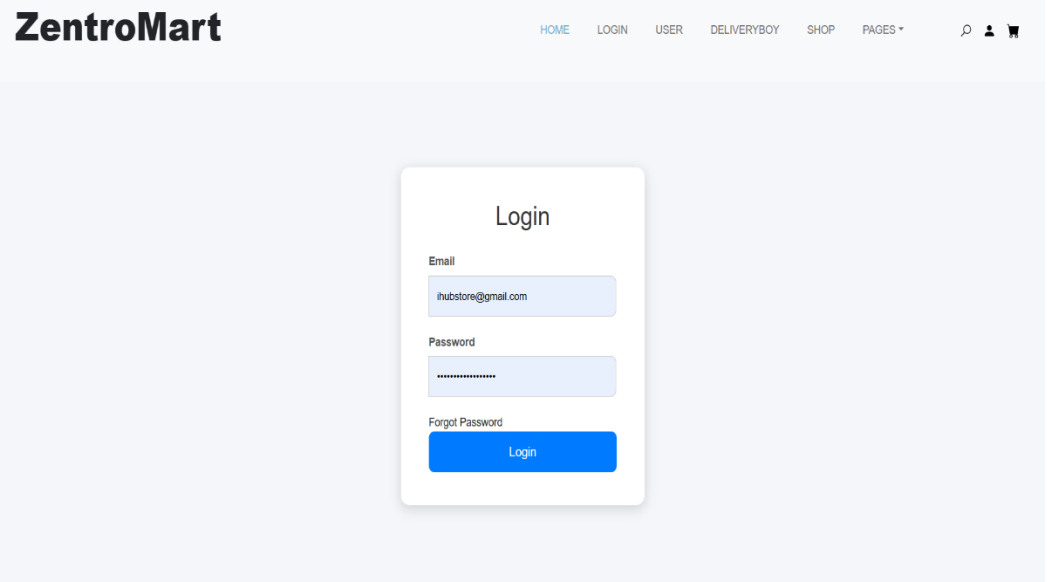
Deliveryboy list:



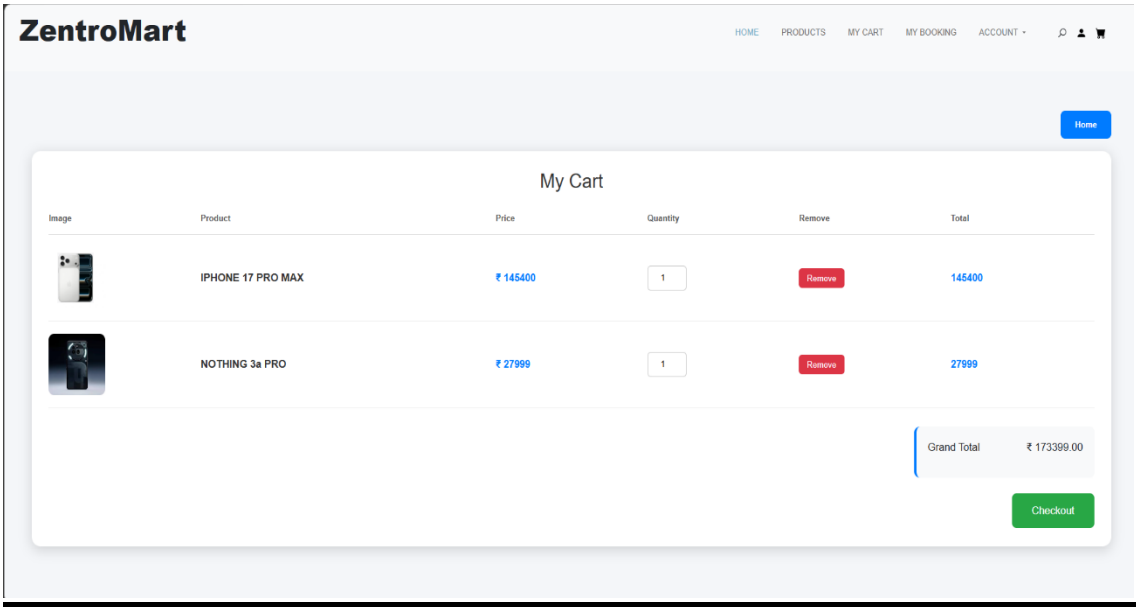
USER Registration:



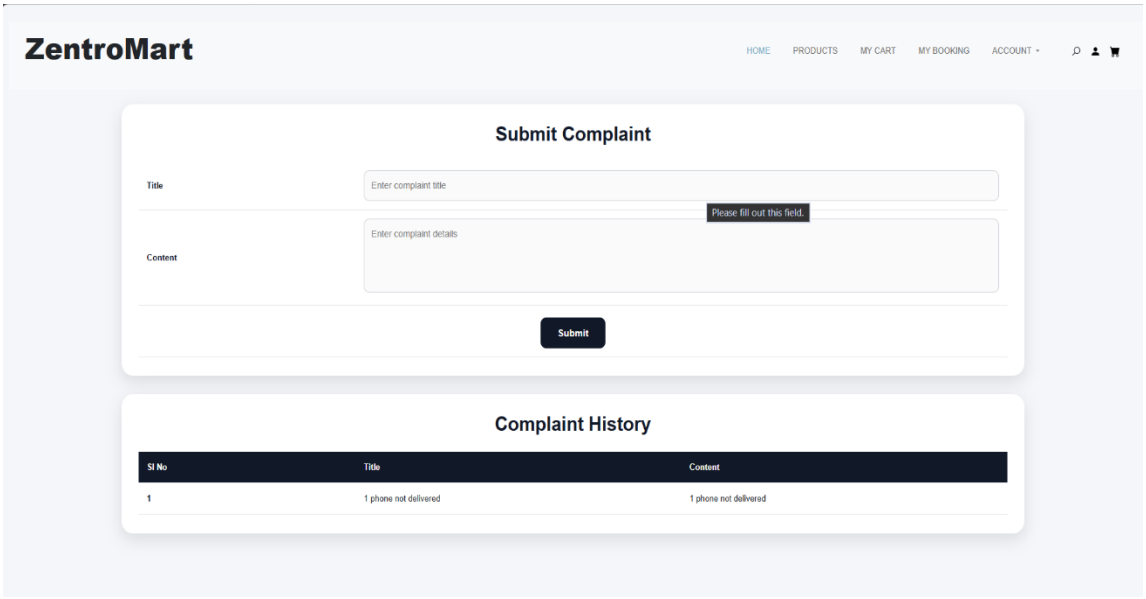
Login:



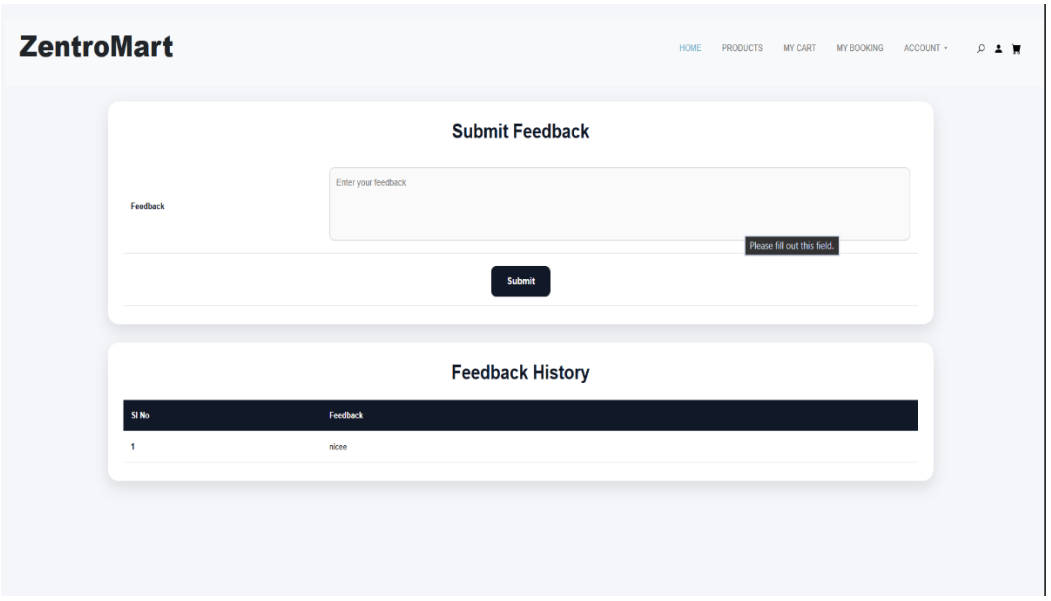
My Cart:



User Complaint:

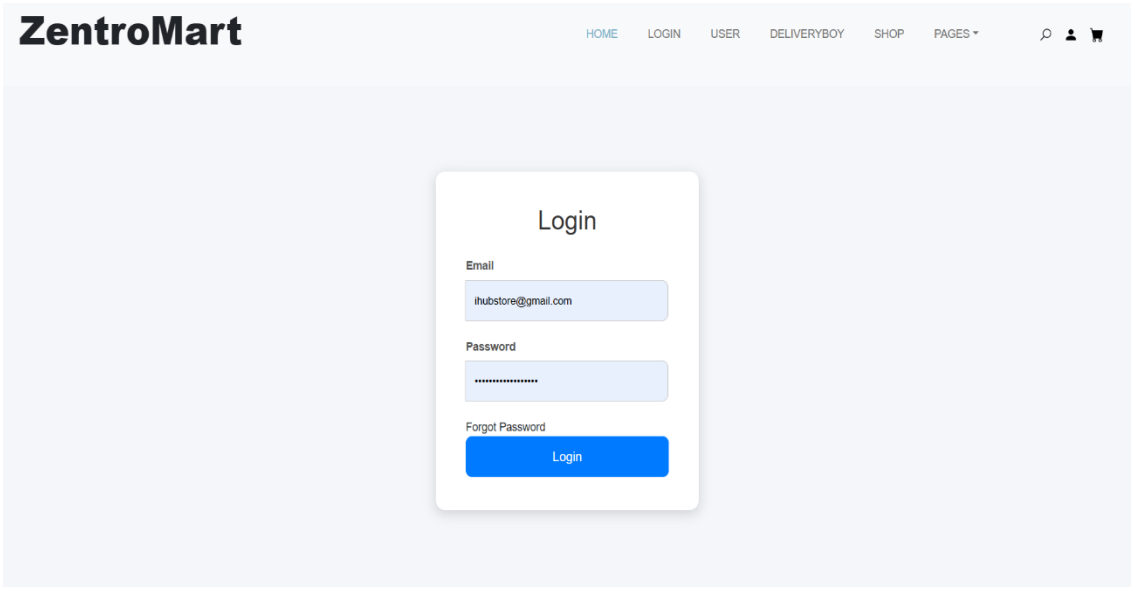


User Feedback:

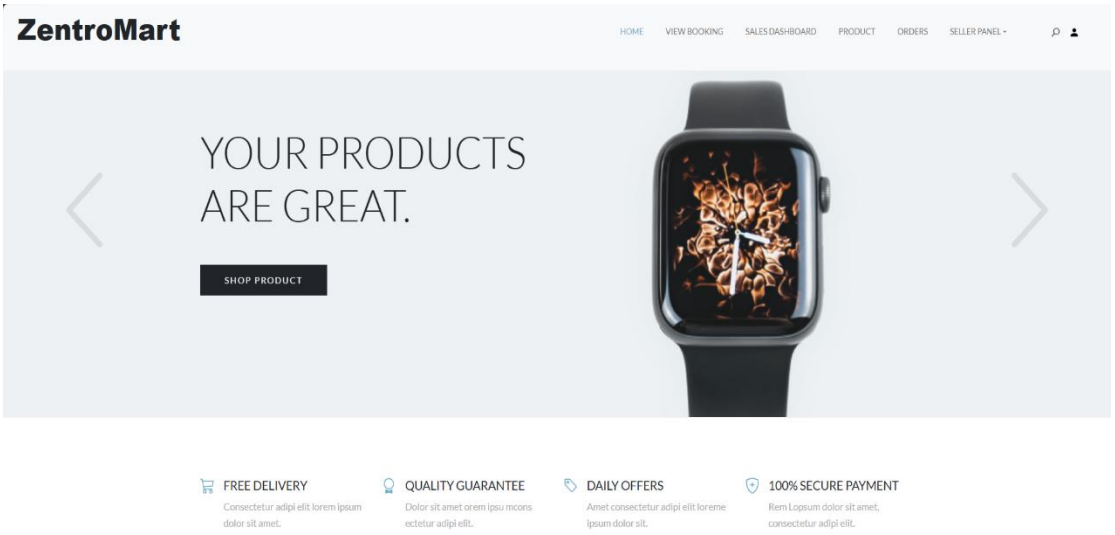


SELLER

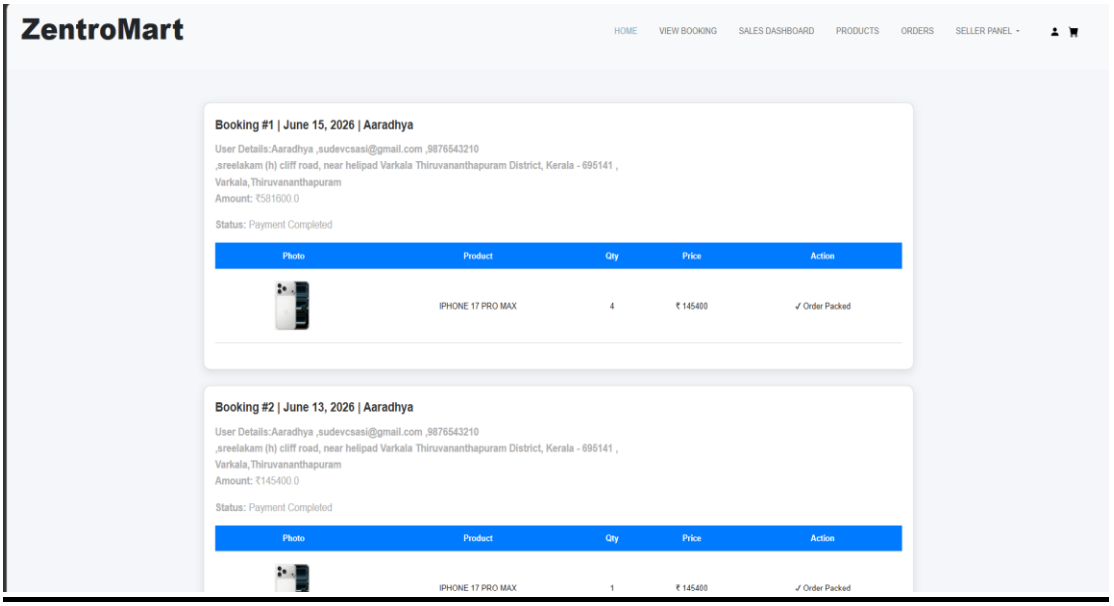
Login:



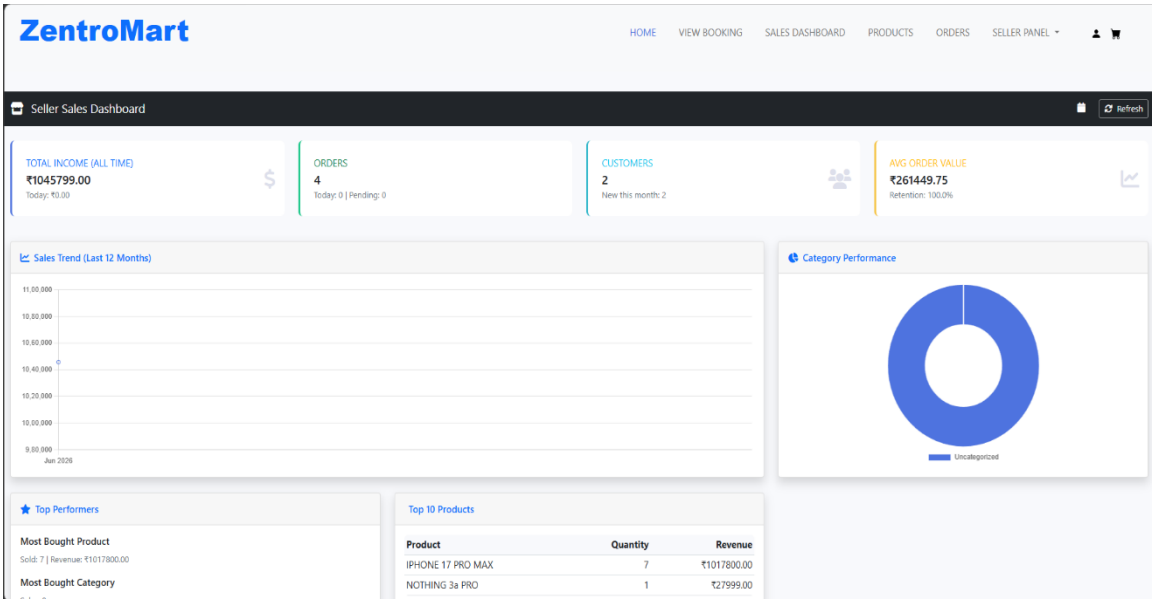
Home Page:



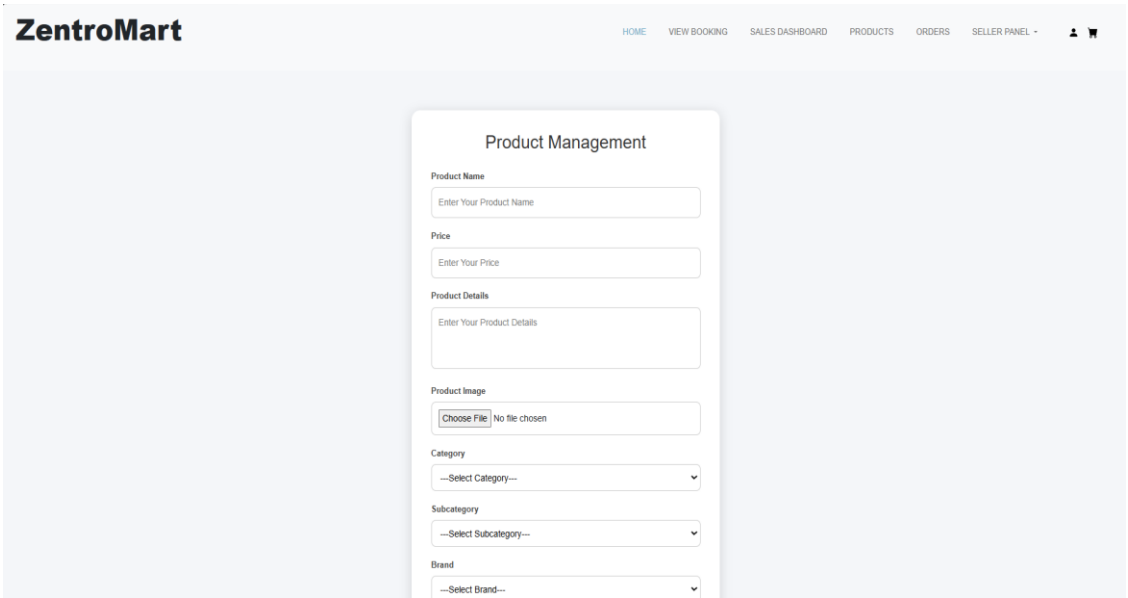
View booking:



Sales Dashboard:

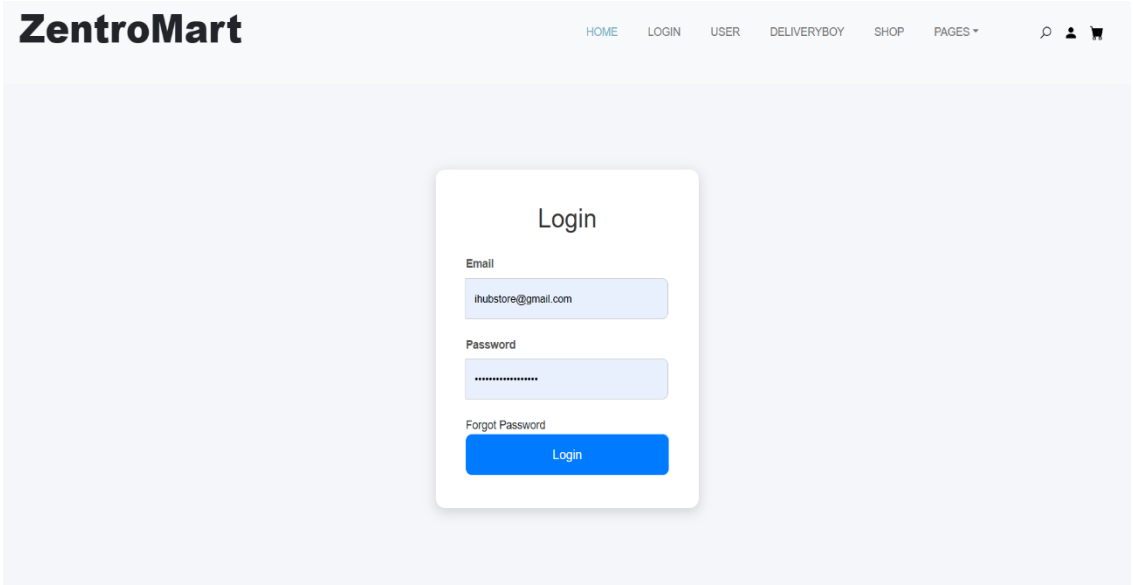


Product Management:

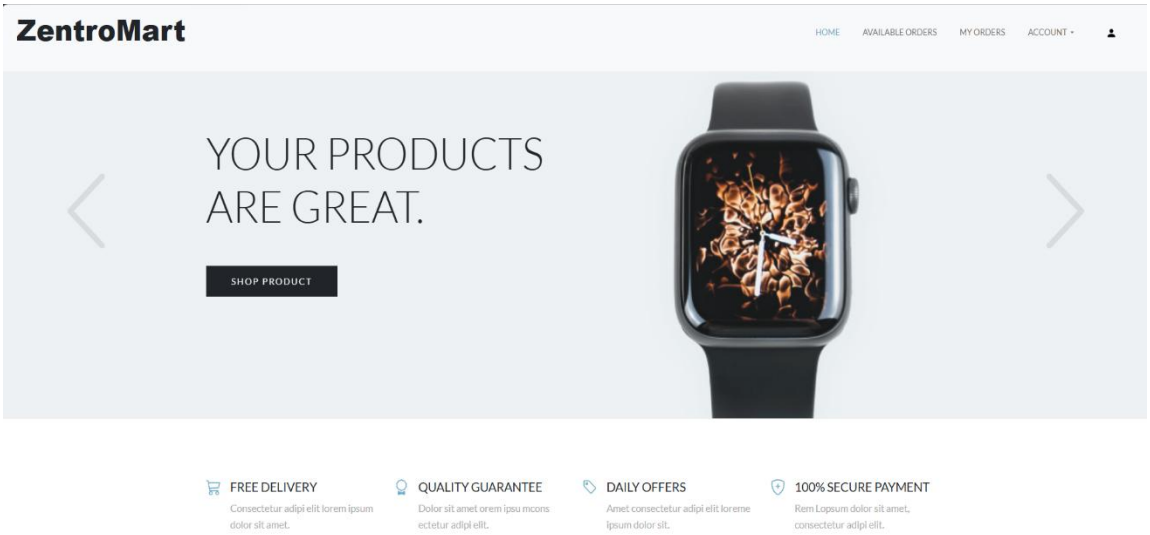


DELIVERYBOY

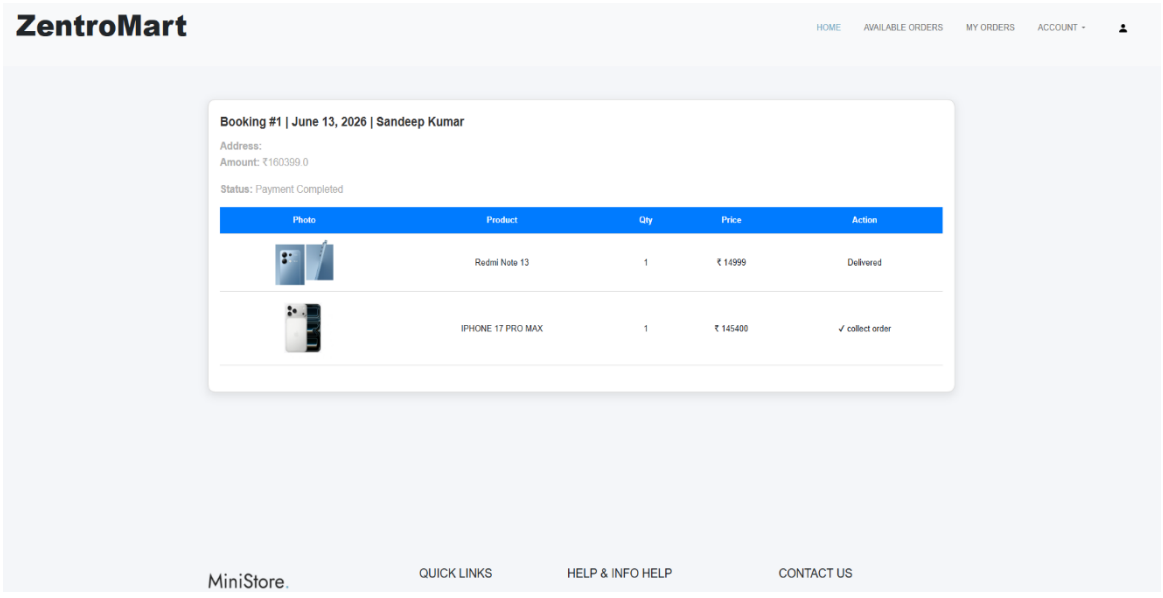
Login:



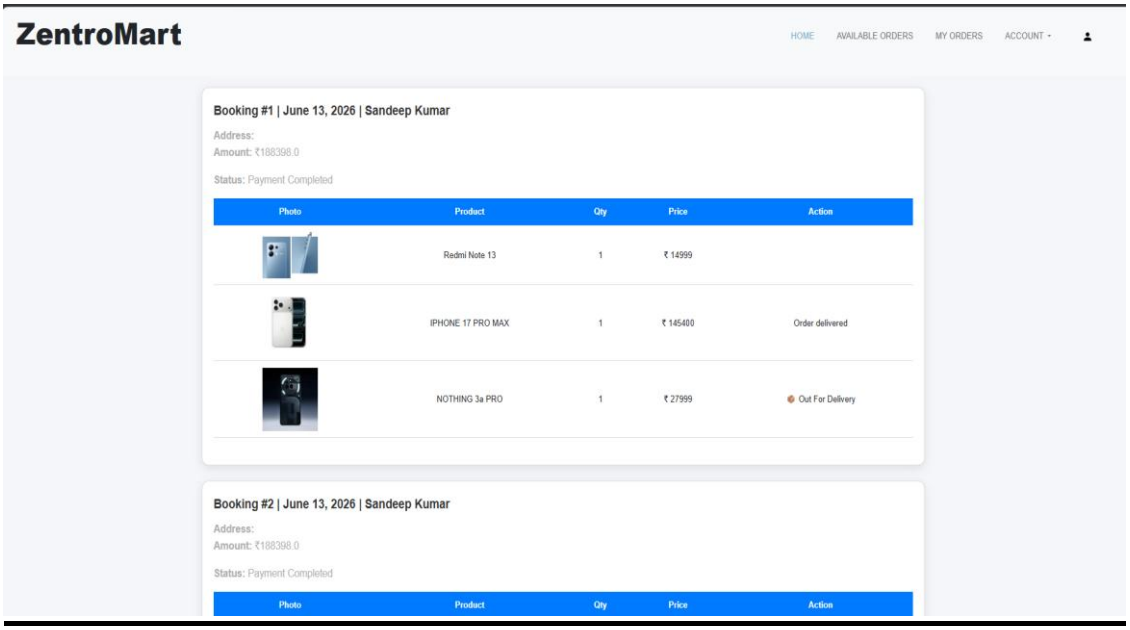
Homepage:



Available Booking:



My Orders:


ZentroMart
HOME
AVAILABLE ORDERS
MY ORDERS
ACCOUNT
Booking #1 | June 13, 2026 | Sandeep Kumar
Address:
Amount: ₹188398.0
Status: Payment Completed
Photo
Product
Qty
Price
Action
Redmi Note 13
1
₹ 14999
IPHONE 17 PRO MAX
1
₹ 145400
Order delivered
NOTHING 3a PRO
1
₹ 27999
Out For Delivery
Booking #2 | June 13, 2026 | Sandeep Kumar
Address:
Amount: ₹188398.0
Status: Payment Completed
Photo
Product
Qty
Price
Action

10.2 CODING

Login:

```
{% extends 'Guest/Header.html' %}
{% load static %}
{% block content %}
<style>
   body{
      background: #f4f6f9;
      font-family: Arial, Helvetica, sans-serif;
   }
   .login-container{
      width: 100%;
      display: flex;
      justify-content: center;
      align-items: center;
      padding: 80px 0;
   }
   .login-box{
      background: #fff;
      padding: 40px;
      border-radius: 12px;
      box-shadow: 0px 4px 15px rgba(0,0,0,0.15);
      width: 350px;
   }
   .login-box h2{
      text-align: center;
      margin-bottom: 25px;
      color: #333;
   }
   .input-group{
      margin-bottom: 20px;
   }
   .input-group label{
      display: block;
      margin-bottom: 6px;
      color: #555;
      font-weight: bold;
   }
      </div>
      <tr>
          <td colspan="2" align="right"><a href="{% url '
Guest:forgotpassword'%}">Forgot Password</a></td>
      </tr>
```

```
    .input-group input{
        width: 100%;
        padding: 12px;
        border: 1px solid #ccc;
        border-radius: 8px;
        font-size: 15px;
        transition: 0.3s;
    }
    .input-group input:focus{
        border-color: #007bff;
        outline: none;
        box-shadow: 0px 0px 5px rgba(0,123,255,0.5);
    }
    .login-btn{
        width: 100%;
        padding: 12px;
        border: none;
        border-radius: 8px;
        background: #007bff;
        color: white;
        font-size: 16px;
        cursor: pointer;
        transition: 0.3s;
    }
    .login-btn:hover{
        background: #0056b3;
    }
    @media(max-width: 400px){
        .login-box{
            width: 90%;
            padding: 25px;
        }
    }
</style>

<div class="login-container">
    <form method="post" class="login-box">
        {% csrf_token %}
<h2>Login</h2>

        <div class="input-group">
            <label>Email</label>
            <input type="text" name="txt_login"
                placeholder="Enter Your Email" required>
        </div>
```

```
      <div class="input-group">
         <label>Password</label>
         <input type="password" name="txt_password"
              placeholder="Enter your Password"
              required pattern="(?=.*\d)(?=.*[a-z])(?=.*[A-Z]).{8,}"
               title="Must contain at least one number and one uppercase and lowercase letter,
     and at least 8 or more characters">
<button type="submit" class="login-btn">Login</button>
  </form>
</div>
{% if msg %}
<script>
  alert("{{msg}}")
  window.location="{% url 'Guest:login' %}"
</script>
{% endif %}
{% endblock content %}
```

Views:

```
def login(request):
   if request.method=="POST":
         email=request.POST.get("txt_login")
         password=request.POST.get("txt_password")
         admincount=tbl_adminregistration.objects.filter(adminregistration_email=email,
     adminregistration_password=password).count()
     usercount=tbl_user.objects.filter(user_email=email,user_password=password).count()
    sellercount=tbl_shop.objects.filter(shop_email=email,shop_password=password).count()
   dboycount=tbl_dboy.objects.filter(dboy_email=email,dboy_password=password).count()
if admincount>0:
admin=tbl_adminregistration.objects.get(adminregistration_email=email,adminregistration_
password=password)
request.session['aid']=admin.id
return redirect("Admin:Homepage")
elif usercount>0:
user=tbl_user.objects.get(user_email=email,user_password=password)
if user.user_status == 0:
return render(request,"Guest/Login.html",{'msg':"Registration is pending"})
elif user.user_status == 2:
 return render(request,"Guest/Login.html",{'msg':"Registration is rejected"})
 else:
 request.session['uid']=user.id
 return redirect("User:Homepage")
 elif sellercount>0:
seller=tbl_shop.objects.get(shop_email=email,shop_password=password)
```

```
 if seller.shop_status == 0:
  return render(request,"Guest/Login.html",{'msg':"Registration is pending"})
  elif seller.shop_status == 2:
  return render(request,"Guest/Login.html",{'msg':"Registration is rejected"})
  else:
   request.session['sid']=seller.id
   return redirect("Seller:Homepage")
   elif dboycount>0:
   dboy=tbl_dboy.objects.get(dboy_email=email,dboy_password=password)
   if dboy.dboy_status == 0:
   return render(request,"Guest/Login.html",{'msg':"Registration is pending"})
  elif dboy.dboy_status == 2:
  return render(request,"Guest/Login.html",{'msg':"Registration is rejected"})
  else:
   request.session['bid']=dboy.id
   return redirect("dboy:Homepage")
    else:
   return render(request,"Guest/Login.html")
     else:
   return render(request,"Guest/Login.html")
```

User Register

```
{% extends 'Guest/Header.html' %}
{% block content %}
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>New User Registration</title>

  <style>
     * {
        margin: 0;
        padding: 0;
        box-sizing: border-box;
        font-family: 'Poppins', sans-serif;
     }

     body {
        background: #f4f6f9;
     }
```

```
    .register-container {
    width: 100%;
    display: flex;
    justify-content: center;
    align-items: center;
    padding: 50px 20px;
}

.register-card {
    width: 50%;
    background: #fff;
    padding: 35px;
    border-radius: 18px;
    box-shadow: 0 10px 25px rgba(0, 0, 0, 0.1);
}

.register-card h1 {
    text-align: center;
    margin-bottom: 30px;
    color: #111827;
    font-size: 36px;
    font-weight: 600;
}

table {
    width: 100%;
}

td {
    padding: 12px;
    vertical-align: top;
}

td:first-child {
    width: 30%;
    font-weight: 600;
    color: #333;
}
```

```
input[type="text"],
    input[type="password"],
    input[type="file"],
    select,
    textarea {
        width: 100%;
        padding: 12px;
        border: 1px solid #d1d5db;
        border-radius: 10px;
        outline: none;
        font-size: 15px;
        transition: 0.3s;
        background: #fafafa;
    }

    input[type="text"]:focus,
    input[type="password"]:focus,
    select:focus,
    textarea:focus {
        border-color: #2563eb;
        box-shadow: 0 0 5px rgba(37, 99, 235, 0.4);
        background: #fff;
    }

    textarea {
        resize: none;
        height: 100px;
    }

    .gender-box {
        display: flex;
        gap: 15px;
        align-items: center;
        margin-top: 8px;
    }

    .gender-box label {
        font-weight: 500;
        color: #444;
    }

    .btn-box {
        text-align: center;
        padding-top: 20px;
    }
```

```
.btn-box input {
        padding: 12px 28px;
        border: none;
        border-radius: 10px;
        cursor: pointer;
        font-size: 15px;
        font-weight: 600;
        transition: 0.3s;
        margin: 5px;
    }

    .submit-btn {
        background: #111827;
        color: white;
    }

    .submit-btn:hover {
        background: #2563eb;
    }

    .reset-btn {
        background: #e5e7eb;
        color: #111827;
    }

    .reset-btn:hover {
        background: #d1d5db;
    }

    @media(max-width: 992px) {
        .register-card {
            width: 70%;
        }
    }

    @media(max-width: 768px) {
        .register-card {
            width: 100%;
            padding: 25px;
        }

        .register-card h1 {
            font-size: 28px;
        }

        td {
            display: block;
            width: 100%;
        }
```

```
        td:first-child {
            padding-bottom: 5px;
        }
    }
</style>

</head>

<body>

<div class="register-container">
    <div class="register-card">

        <h1>New User</h1>

        <form method="POST" enctype="multipart/form-data">
            {% csrf_token %}

            <table>

                <tr>
                    <td>District</td>
                    <td>
                        <select name="sel_district" id="sel_district" required>
                            <option value="">--- Select District ---</option>

                            {% for i in district %}
                            <option value="{{i.id}}">
                                {{i.district_name}}
                            </option>
                            {% endfor %}

                        </select>
                    </td>
                </tr>

                <tr>
                    <td>Place</td>
                    <td>
                        <select name="sel_place" id="sel_place" required>
                            <option value="">--- Select Place ---</option>
                        </select>
                    </td>
                </tr>
```

```
<tr>
                <td>Name</td>
                <td>
                   <input type="text" name="txt_name" required
                      placeholder="Enter your name"
                      minlength="5"
                      title="Name Allows Only Alphabets, Spaces and First Letter Must Be
Capital Letter"
                      pattern="^[A-Z]+[a-zA-Z ]*$" />
                </td>
             </tr>

             <tr>
                <td>Gender</td>
                <td>
                   <div class="gender-box">

                      <label>
                         <input type="radio" name="rd_gender" value="Male" required>
                         Male
                      </label>

                      <label>
                         <input type="radio" name="rd_gender" value="Female">
                         Female
                      </label>

                      <label>
                         <input type="radio" name="rd_gender" value="Others">
                         Others
                      </label>

                   </div>
                </td>
             </tr>

             <tr>
                <td>Contact</td>
                <td>
                   <input type="text" name="txt_contact"
                      required
                      placeholder="Enter your phone number"
                      pattern="[7-9]{1}[0-9]{9}"
                      title="Phone number must start with 7-9 and contain 10 digits" />
                </td>
             </tr>
```

```
<tr>
                <td>Email</td>
                <td>
                    <input type="text" name="txt_email"
                        required
                        placeholder="Enter your email">
                </td>
            </tr>

            <tr>
                <td>Password</td>
                <td>
                    <input type="password"
                        name="txt_pwd"
                        required
                        placeholder="Enter your password"
                        pattern="(?=.*\d)(?=.*[a-z])(?=.*[A-Z]).{8,}"
                        title="Must contain at least one number and one uppercase and
                         lowercase letter, and at least 8 or more characters" />
                </td>
            </tr>

            <tr>
                <td>Address</td>
                <td>
                    <textarea name="txt_address"
                        required
                        placeholder="Enter your address"></textarea>
                </td>
            </tr>

            <tr>
                <td>Photo</td>
                <td>
                    <input type="file" name="file_photo" accept="image/*" required>
                </td>
            </tr>

            <tr>
                <td>Proof</td>
                <td>
                    <input type="file" name="file_proof" accept="image/*" required>
                </td>
            </tr>
```

```
<tr>
                <td colspan="2" class="btn-box">
              <input type="submit" name="btn_submit" value="Submit" class="submit-btn">

                  <input type="reset" name="btn_cancel" value="Cancel" class="reset-btn">
                </td>
              </tr>

            </table>

        </form>

    </div>
  </div>

</body>

</html>

<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.6.0/jquery.min.js"></script>

<script>
  $("#sel_district").change(function () {

      var did = $(this).val();

      var ur = "{% url 'Guest:ajaxplace' %}"

      $.ajax({
          url: ur,
          data: {
              did: did,
          },
          success: function (data) {
              $("#sel_place").html(data);
          },
      });

  });
</script>

{% endblock content %}
```

VIEWS:

```
def User(request):
district=tbl_district.objects.all()
if request.method == "POST":
    name=request.POST.get("txt_name")
    gender=request.POST.get("rd_gender")
    contact=request.POST.get("txt_contact")
    email=request.POST.get("txt_email")
    address=request.POST.get("txt_address")
    password=request.POST.get("txt_pwd")
    photo=request.FILES.get("file_photo")
    proof=request.FILES.get("file_proof")
    placeid=tbl_place.objects.get(id=request.POST.get("sel_place"))

    tbl_user.objects.create(
        user_name=name,
        user_gender=gender,
        user_contact=contact,
        user_email=email,
        user_address=address,
        user_password=password,
        user_photo=photo,
        user_proof=proof,
        place=placeid
    )
    return redirect("Guest:User")
else:
    return render(request,"Guest/NewUser.html",{'district':district})
```

Product

```
{% extends 'Seller/Header.html' %}
{% load static %}
{% block content %}

<style>

  body{
    background: #f4f6f9;
    font-family: Arial, Helvetica, sans-serif;
  }

  .container{
    width: 100%;
    padding: 40px 0;
  }
```

```
.form-box{
    width: 550px;
    margin: auto;
    background: #fff;
    padding: 35px;
    border-radius: 15px;
    box-shadow: 0px 5px 20px rgba(0,0,0,0.15);
}

.form-box h2{
    text-align: center;
    margin-bottom: 25px;
    color: #333;
    font-size: 30px;
}

.form-group{
    margin-bottom: 18px;
}

.form-group label{
    display: block;
    margin-bottom: 7px;
    font-weight: bold;
    color: #444;
}

.form-group input,
.form-group textarea,
.form-group select{
    width: 100%;
    padding: 12px;
    border: 1px solid #ccc;
    border-radius: 8px;
    font-size: 15px;
    transition: 0.3s;
    box-sizing: border-box;
}

.form-group textarea{
    resize: none;
    height: 100px;
}
```

```
.form-group input:focus,
  .form-group textarea:focus,
  .form-group select:focus{
     border-color: #007bff;
     outline: none;
     box-shadow: 0px 0px 6px rgba(0,123,255,0.4);
  }

  .btn-group{
     display: flex;
     justify-content: space-between;
     margin-top: 25px;
  }

  .btn-submit,
  .btn-reset{
     width: 48%;
     padding: 12px;
     border: none;
     border-radius: 8px;
     font-size: 16px;
     cursor: pointer;
     transition: 0.3s;
  }

  .btn-submit{
     background: #007bff;
     color: white;
  }

  .btn-submit:hover{
     background: #0056b3;
  }

  .btn-reset{
     background: #dc3545;
     color: white;
  }

  .btn-reset:hover{
     background: #a71d2a;
  }
```

```
.table-box{
    width: 95%;
    margin: 40px auto;
    background: #fff;
    padding: 25px;
    border-radius: 15px;
    box-shadow: 0px 5px 20px rgba(0,0,0,0.15);
    overflow-x: auto;
}

.table-box h2{
    text-align: center;
    margin-bottom: 20px;
    color: #333;
}

table{
    width: 100%;
    border-collapse: collapse;
}

table th,
table td{
    padding: 12px;
    text-align: center;
    border: 1px solid #ddd;
}

table th{
    background: #007bff;
    color: white;
}

table tr:nth-child(even){
    background: #f2f2f2;
}

.product-img{
    width: 120px;
    height: 120px;
    object-fit: cover;
    border-radius: 10px;
}
```

```
.action-btn{
    text-decoration: none;
    padding: 7px 12px;
    border-radius: 5px;
    color: white;
    font-size: 14px;
    margin: 2px;
    display: inline-block;
}

.delete-btn{
    background: #dc3545;
}

.edit-btn{
    background: #28a745;
}

.gallery-btn{
    background: #17a2b8;
}

.stock-btn{
    background: #ffc107;
    color: black;
}

.delete-btn:hover{
    background: #a71d2a;
}

.edit-btn:hover{
    background: #1e7e34;
}

.gallery-btn:hover{
    background: #11707f;
}

.stock-btn:hover{
    background: #d39e00;
}

@media(max-width: 700px){
```

```
  .form-box{
        width: 90%;
        padding: 25px;
    }

    .btn-group{
        flex-direction: column;
    }

    .btn-submit,
    .btn-reset{
        width: 100%;
        margin-bottom: 10px;
    }
  }

</style>

<div class="container">

<form method="POST" enctype="multipart/form-data">

  {% csrf_token %}

  <div class="form-box">

    <h2>Product Management</h2>

    <div class="form-group">
      <label>Product Name</label>
      <input type="text"
          name="txt_product"
          value="{{productdata.product_name}}"
          required
          placeholder="Enter Your Product Name">
    </div>

    <div class="form-group">
      <label>Price</label>
      <input type="text"
          name="txt_price"
          value="{{productdata.product_price}}"
          required
          placeholder="Enter Your Price">
    </div>
```

```
    <div class="form-group">
            <label>Product Details</label>
            <textarea name="details"
                    required
                    placeholder="Enter Your Product
Details">{{productdata.product_details}}</textarea>
        </div>

        <div class="form-group">
            <label>Product Image</label>
            <input type="file"
                name="photo"
                accept="image/*"
                required>
        </div>

        <div class="form-group">
            <label>Category</label>
            <select name="sel_category" id="sel_category" required>

                <option value="">---Select Category---</option>

                {% for i in category %}

                <option value="{{i.id}}"
                {% if i.id == editdata.Category_name_id %} selected {% endif %}>
                    {{ i.Category_name }}
                </option>

                {% endfor %}

            </select>
        </div>

        <div class="form-group">
            <label>Subcategory</label>
            <select name="sel_subcategory" id="sel_subcategory" required>
                <option value="">---Select Subcategory---</option>
            </select>
        </div>

        <div class="form-group">
            <label>Brand</label>
            <select name="sel_br" required>

                <option value="">---Select Brand---</option>

                {% for i in brand %}
```

```
<option value="{{i.id}}"
            {% if i.id == editdata.brand_name_id %} selected {% endif %}>
               {{ i.brand_name }}
            </option>

            {% endfor %}

         </select>
      </div>

      <div class="btn-group">
         <input type="submit" value="Submit" class="btn-submit">
         <input type="reset" value="Cancel" class="btn-reset">
      </div>

   </div>

   <div class="table-box">

      <h2>Product List</h2>

      <table>

         <tr>
            <th>SL NO</th>
            <th>Product</th>
            <th>Price</th>
            <th>Details</th>
            <th>Image</th>
            <th>Category</th>
            <th>Subcategory</th>
            <th>Brand</th>
            <th>Action</th>
         </tr>

         {% for i in product %}

         <tr>

            <td>{{ forloop.counter }}</td>
            <td>{{i.product_name}}</td>
            <td>{{i.product_price}}</td>
            <td>{{i.product_details}}</td>

            <td>
               <img src="{{i.product_photo.url}}"
                  class="product-img">
            </td>
```

```
<td>{{i.subcategory.Category_name.Category_name}}</td>
            <td>{{i.subcategory.subcategory_name}}</td>
            <td>{{i.brand.brand_name}}</td>

            <td>

               <a href="{% url 'Seller:productdelete' i.id %}"
                  class="action-btn delete-btn">
                  Delete
               </a>

               <a href="{% url 'Seller:productedit' i.id %}"
                  class="action-btn edit-btn">
                  Edit
               </a>

               <a href="{% url 'Seller:addgallery' i.id %}"
                  class="action-btn gallery-btn">
                  Gallery
               </a>

               <a href="{% url 'Seller:addstock' i.id %}"
                  class="action-btn stock-btn">
                  Stock
               </a>

            </td>

         </tr>

         {% endfor %}

      </table>

   </div>

</form>

</div>

<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.6.0/jquery.min.js"></script>

<script>
```

```
$("#sel_category").change(function () {
    var did = $(this).val();
    var ur = "{% url 'Seller:ajaxsubcategory' %}";
    $.ajax({
      url: ur,
      data: { did: did },
      success: function (data) {
        $("#sel_subcategory").html(data);
      },
    });
  });
</script>
{% endblock content %}
```

Views

```
def product(request):
    product_data=tbl_product.objects.filter(shop=request.session['sid'])
    category=tbl_category.objects.all()
    brands=tbl_brand.objects.all()
    if request.method=="POST":
        name=request.POST.get("txt_product")
        details=request.POST.get("details")
        photo=request.FILES.get("photo")
        price=request.POST.get("txt_price")
        subcategoryid=tbl_subcategory.objects.get(id=request.POST.get("sel_subcategory"))
        brandid=tbl_brand.objects.get(id=request.POST.get("sel_br"))
        shopid=tbl_shop.objects.get(id=request.session['sid'])
        tbl_product.objects.create(
            product_name=name,
            product_details=details,
            product_price=price,
            product_photo=photo,
            subcategory=subcategoryid,
            shop=shopid,
            brand=brandid
)
        return redirect("Seller:product")
    else:
        return render(request,"seller/Product.html",{'brand':brands,'category':category,
     'product':product_data)
```